

Discovering Domain-Driven Design

Business rules, TDD, and Clean Architecture

A running-example learning journey from a school calendar to an architectural model.

Student edition

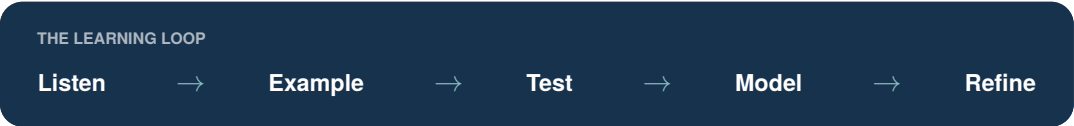
THE THREAD

Listen to the business → Test the rule → Discover the model → Protect it

START HERE

One school. One problem. 44 decisions.

This book begins with a school trying to build a trustworthy academic calendar. Technical ideas appear only when the story creates a reason for them.



BUSINESS

Start with the rule

Clarify what must be true before choosing technical structure.

MODEL

Discover responsibility

Let examples reveal value, identity, consistency, and collaboration.

ARCHITECTURE

Protect the model

Add boundaries only when the running system actually needs them.

Not a pattern catalogue. Not a folder recipe. Not a code tour. The sequence is cumulative: each step should make the next abstraction feel necessary.

Running context SchoolGrid / Academic calendar / 2026 teaching scenario

YOUR ROUTE THROUGH SCHOOLGRID

The learning map

Every step answers a question created by the step before it. Concepts sit underneath the step where they become useful.

TRACK 01 / Start with the business

01

The School Calendar Problem
Problem framing · scope · acceptable calendar

p. 1

02

Discovering Business Rules
Rules · assumptions · boundary cases

p. 5

03

From Examples to Tests
TDD · red-green-refactor · observable behaviour

p. 12

04

Domain, Experts, and Ubiquitous Language
Domain · domain expert · ubiquitous language

p. 18

05

From Rules to a Domain Model
Model · responsibility · behaviour over data

p. 25

TRACK 02 / Build the smallest domain concepts

06

Value Objects
DateRange · equality by value · invariant

p. 34

07

Value Object Behaviour
Immutability · duration · inclusion · overlap

p. 41

08

Making Invalid Values Impossible
TermNumber · constrained values · primitives

p. 50

09	Entities: When Identity Matters Entity · identity · continuity	p. 58
10	Entity State and Behaviour State transition · encapsulation · intention-revealing methods	p. 66
TRACK 03 / Discover aggregates through pressure		
11	When Valid Objects Become Invalid Together Cross-object rule · consistency problem	p. 76
12	Aggregates Aggregate · consistency boundary	p. 85
13	The Aggregate Root Aggregate root · controlled access	p. 93
14	Aggregate Invariants Uniqueness · ordering · non-overlap · calendar boundary	p. 102
15	Transactional Consistency Atomic change · transaction boundary · concurrency question	p. 113
TRACK 04 / Complete the domain model		
16	Domain Services Domain service · stateless domain operation · misplaced behaviour	p. 114
17	Factories Creation policy · valid construction · factory tradeoff	p. 115
18	Domain Events Past-tense fact · domain event · side effects	p. 116
19	Repositories Aggregate persistence · repository abstraction	p. 117
20	Repository Boundaries Aggregate root repository · DAO contrast · projections	p. 118
TRACK 05 / Move from domain to use cases		
21	A User Wants to Schedule a Term Use case · orchestration · request flow	p. 119
22	Application Services Application service · coordination · transaction orchestration	p. 120
23	Commands and Input Models Command · input model · intent	p. 121
24	DTOs DTO · boundary data · domain-object separation	p. 122
25	Queries and Read Models Query model · projection · read optimisation	p. 123
TRACK 06 / Introduce Clean Architecture naturally		
26	The Architecture Problem Technical pressure · policy protection	p. 124
27	Clean Architecture Dependency rule · policy centre · boundaries	p. 125

28	The Domain Layer Domain model · enterprise rules	p. 126
29	The Application Layer Use cases · orchestration · ports	p. 127
30	The Infrastructure Layer Adapters · persistence · external systems	p. 128
31	The Presentation Layer Inbound adapter · request translation · response mapping	p. 129
TRACK 07 / Control dependency direction		
32	The Dependency Rule Source dependency · runtime flow · inward direction	p. 130
33	Dependency Inversion Through the Repository Abstraction · injection · composition root	p. 131
34	Why the Domain Does Not Know the Database Persistence ignorance · mapping · Active Record contrast	p. 132
35	Walking Through the Entire Request End-to-end use case · success · rejection · commit failure	p. 133
TRACK 08 / TDD across the architecture		
36	Testing Value Objects Boundary tests · properties · focused feedback	p. 134
37	Testing Entities and Aggregates Invariant tests · state preservation · identity	p. 135
38	Testing Application Use Cases Fake repository · test doubles · orchestration	p. 136
39	Testing Infrastructure Separately Integration test · transaction · concurrency · mapping	p. 137
40	Different Tests Answer Different Questions Test boundary · risk · confidence	p. 138
TRACK 09 / Put the whole system together		
41	One Business Rule Through Every Layer Rule traceability · authoritative policy · feedback	p. 139
42	DDD vs Clean Architecture vs TDD Model meaning · dependency design · development feedback	p. 140
43	Common Misunderstandings Pattern checklist traps · repository myths · service confusion	p. 141
44	The Complete SchoolGrid Mental Model Synthesis · tradeoffs · model evolution	p. 142

STEP 1

The School Calendar Problem

What you will discover

Explain why SchoolGrid needs more than date storage, follow a calendar change from request to decision, and distinguish an established school rule from an assumption.

Before there is software, there is a calendar

Before a new academic year begins, a school publishes dates that people will plan around. Teachers prepare schemes of work, families plan around opening and closing dates, and administrators coordinate activities that depend on the term calendar. Once those dates are circulated, changing them carelessly creates real confusion.

Our fictional school wants to move this work into **SchoolGrid**. The first capability is deliberately small: maintain the dates of the academic year and allow an administrator to change them without leaving the calendar in a state the school considers impossible.

For the 2026 teaching scenario, the school currently has this calendar:

Term	Opens	Closes
Term 1	1 January 2026	28 March 2026
Term 2	1 May 2026	28 July 2026
Term 3	1 September 2026	28 November 2026

These dates are part of the running example, not an official school calendar. At this point, the simplest Python we need is equally modest:

PYTHON

```
1 from datetime import date
2
3 term_2_start = date(2026, 5, 1)
4 term_2_end = date(2026, 7, 28)
```

The code records two dates. That is useful, but it does not yet tell us what the school permits us to do with them.

Then a real change arrives

A few weeks before Term 2, the headteacher asks for an **earlier opening date**. The administrator must decide whether the change can be accepted before a revised calendar is circulated.

DDD CONCEPT

Domain expert

Someone who understands how the work is actually done and can explain the rules the software must respect. Here, the school administrator provides that knowledge.

DOMAIN WORKSHOP

Moving Term 2 earlier

School administrator / Developer

SA School administrator

The headteacher wants Term 2 to open on 15 April instead of 1 May. Its closing date stays 28 July. Can SchoolGrid make that change?

DEV Developer

I can replace the start date. Before I do, what would make an earlier date unacceptable?

SA School administrator

It cannot fall inside Term 1. Term 1 runs through 28 March.

DEV Developer

So if someone entered 20 March, we reject the change?

SA School administrator

Yes. And we keep the existing Term 2 dates. A bad request must not damage the calendar we already have.

DEV Developer

What about 28 March itself?

SA School administrator

That is still a Term 1 day. Term 2 must start after it.

Now the requested change can be represented:

PYTHON

```
1 proposed_start = date(2026, 4, 15)
```

Nothing about that line tells us whether 15 April is acceptable. To answer the school's question, we also need information from the **neighbouring term**:

PYTHON

```
1 term_1_end = date(2026, 3, 28)
2
3 proposed_start > term_1_end    # True
```

The comparison is not a complete scheduling implementation. It simply makes the newly discovered relationship visible: changing Term 2 requires us to consider where Term 1 ends.

Model evolution**INITIAL INTERPRETATION**

Replace one stored date.

REVISED UNDERSTANDING

Decide whether the proposed change is allowed in the calendar it belongs to.

The boundary tells us more than the dates do

The administrator's answer about 28 March settles an important **edge case**. If Term 1 includes its closing day, then these two proposals are different:

PYTHON

```
1 start_on_28_march = date(2026, 3, 28)    # reject
2 start_on_29_march = date(2026, 3, 29)    # clears this rule
```

Python considers both values valid dates. The school distinguishes them because of what those dates mean in the academic calendar.

CONTRAST

Valid Python value

`date(2026, 3, 28)` is a perfectly valid date object.

Acceptable calendar change

Starting Term 2 on 28 March is rejected because that day still belongs to Term 1.

KEY IDEA The software must understand relationships between the school's data, not merely store individually valid values.

Keep the first problem small

SchoolGrid may eventually manage students, teachers, fees, attendance, examinations, and timetables. None of those are needed to answer today's question.

IN THIS SLICE

One academic year, its numbered terms, their opening and closing dates, and requests to change those dates.

OUTSIDE FOR NOW

Students, teachers, subjects, fees, attendance, examinations, lesson timetables, and distribution of the published calendar.

A narrow first slice keeps the **business decision** visible. We can expand the system later when another requirement creates a reason to do so.

Some answers are known; others are still guesses

The conversation gave us one clear rule: Term 2 must begin after Term 1's final day. It did *not* answer every calendar question.

READING CUE

Boundary case

An example at the exact point where an outcome changes. Here, 28 March is rejected while a later date may be acceptable.

MODELLING

Scope

The part of the real-world problem we deliberately choose to model now. Everything else is deferred, not declared unimportant.

READING CUE

Assumption

Something we currently believe but have not confirmed with the people who own the business rule. An assumption must not quietly become policy.

DOMAIN EVIDENCE SO FAR

ESTABLISHED

Term dates can be changed after they have been entered.

Term 2 must start after Term 1's closing day.

A rejected request must leave the accepted calendar unchanged.

ASSUMPTION

All three terms must be configured at the same time.

Every term must finish inside the same calendar year.

Every school using SchoolGrid will always have exactly three terms.

OPEN QUESTION

May an administrator save an academic year while only Term 1 is scheduled?

May Term 3 continue into January of the following year?

Is three terms a permanent SchoolGrid rule or only this school's current arrangement?

This distinction matters. A programmer can implement an **assumption** perfectly and still produce the wrong system.

Try the decision before designing the model

Try it: What do we know, and what must we ask?

Use the original 2026 calendar and consider these requests:

1. Move the opening of Term 2 to 25 March 2026.
2. Move the opening of Term 2 to 29 March 2026.
3. Move the closing of Term 3 to 15 January 2027.

For each request, state whether the information in this step is enough to decide. If it is, give the established reason. If it is not, write the question you would ask the school. Do not invent a rule to fill the gap.

NEXT QUESTION

We now have examples of accepted, rejected, and unresolved changes. How do we turn those examples into precise rules the software can enforce?

Next: Step 2 – Discovering Business Rules

STEP 2

Discovering Business Rules

What you will discover

Turn concrete calendar examples into precise business rules, distinguish a rule from an assumption, and use boundary cases to expose what a vague requirement has not yet said.

At the end of Step 1, we could answer some calendar questions but not all of them. Moving Term 2 to 25 March was clearly wrong because Term 1 was still in progress. Moving it to 29 March cleared that particular conflict. Extending Term 3 into January 2027 remained unresolved.

The next task is not to design classes. It is to replace those partial answers with rules precise enough that two people looking at the same proposal should reach the same decision.

Start with the smallest relationship

Before discussing three terms together, the developer asks about the two dates that describe a single period. A period can be wrong even when no other term is involved.

DOMAIN WORKSHOP

Can a period be wrong by itself?

School administrator / Developer

DEV Developer

Suppose someone enters 20 May as the opening date and 1 May as the closing date. No other term is involved yet. Should SchoolGrid accept it?

SA School administrator

M-2.1 No. A term cannot close before it opens.

DEV Developer

What if the opening and closing dates are both 20 May?

SA School administrator

M-2.2 That is not backwards. Do not reject it merely because the two dates are equal.

That answer gives us our first general rule.

Rule BR-2.1: A period cannot run backwards

For a proposed period, the closing date must be **on or after** the opening date.

Business evidence: M-2.1 and M-2.2.

BUSINESS RULE

Business rule

A statement about what the business permits, requires, or forbids. It should be precise enough to decide concrete examples consistently.

BUSINESS LANGUAGE**Inclusive boundary**

A boundary date belongs to the period itself. If the start and end are the same date, the period contains that one day.

The wording matters. “After” would reject equal dates; “on or after” does not. At this level, the two boundaries are **inclusive**: the opening day and closing day both belong to the period.

Example: Three ways to probe the chronology rule

Given we are checking only the relationship between a proposed opening and closing date.

When the dates are 1 May–28 July, 20 May–1 May, and 20 May–20 May.

Then the first is chronological, the second violates BR-2.1, and the third does not violate that rule.

A same-day period passing this rule does *not* prove that a one-day school term is sensible. It proves only that equality is not a chronology error. If the school later introduces a minimum term length, that would be a different rule.

CONTRAST**A rule**

The closing date must be on or after the opening date.

An example

20 May–1 May is rejected because it violates that rule.

The rule is general. The example gives us evidence that we have understood how the rule behaves at a particular point.

Now settle what a term number means

The first calendar happened to show Terms 1, 2 and 3. Step 1 deliberately did not turn that observation into a rule. The developer now asks directly.

DOMAIN WORKSHOP**Do all three terms have to exist immediately?**

School administrator / Developer

DEV Developer

For this SchoolGrid calendar, what term numbers are allowed?

SA School administrator

M-2.3 Term 1, Term 2 and Term 3. Those are the three positions in our academic year.

DEV Developer

Must I enter all three before I can save the year?

SA School administrator

M-2.4 No. We normally configure the year progressively. We may enter Term 1 first and add the others as their dates are confirmed.

DEV Developer

And if Term 2 already exists, may I add another Term 2 with different dates?



School administrator

M-2.5 No. Change the existing Term 2. Do not create a second one.

Three distinct decisions have appeared. They should not be collapsed into the vague statement “the school has three terms.”

Rule BR-2.2: Only the three term numbers are valid

In this SchoolGrid teaching scenario, a term number must be **1, 2, or 3**.

Business evidence: M-2.3.

Rule BR-2.3: A term number occurs at most once

Within one academic year, there may not be two scheduled terms with the same term number.

Business evidence: M-2.5.

Rule BR-2.4: The year may be configured progressively

An academic year does **not** need all three terms scheduled at every moment. The administrator may save the year while only part of the calendar has been configured.

Business evidence: M-2.4.

This is an important distinction. “Only 1, 2 and 3 are valid term numbers” is not the same statement as “all three must already exist.”

Example: A calendar still being prepared

Given Academic Year 2026 currently contains only Term 1.

When the administrator saves the calendar before Term 2 and Term 3 dates have been confirmed.

Then the year is not rejected merely because two permitted term numbers have not yet been scheduled.

MODELLING

Progressive configuration

A valid working state may be incomplete because the business intentionally adds information over time. Incomplete does not automatically mean invalid.

A term must also fit the year around it

We can now return to the unresolved request from Step 1: Term 3 opens on 1 September 2026 but is changed to close on 15 January 2027.

DOMAIN WORKSHOP

Can a term cross into the next calendar year?

School administrator / Developer



Developer

If Academic Year 2026 has a Term 3 that closes on 15 January 2027, should SchoolGrid accept it?

SA School administrator

M-2.6 Not in this system. When we configure the 2026 academic year, its term dates must stay in 2026.

DEV Developer

Is that a universal rule for every school?

SA School administrator

No. It is the policy we are using for this SchoolGrid calendar. Do not generalise it beyond that.

That last sentence is as important as the rule itself. Domain rules belong to a particular business context; we should not quietly promote a local policy into a law of schooling.

Rule BR-2.5: Scheduled term dates stay inside the academic year's calendar year

For the SchoolGrid teaching policy, every scheduled opening and closing date for Academic Year 2026 must fall within calendar year **2026**.

Business evidence: M-2.6; this is a local SchoolGrid policy.

Example: Crossing the year boundary

Given Term 3 opens on 1 September 2026.

When its proposed closing date is 15 January 2027.

Then the proposal is rejected by BR-2.5.

Valid periods can still form an invalid calendar

We now know how to judge a period by itself and how to keep it inside the named year. Neither rule tells us how two different terms may sit beside each other.

Step 1 already exposed one boundary: Term 1 closes on 28 March, and Term 2 may not begin on that same date. The developer pushes the idea further.

DOMAIN WORKSHOP

How must the terms relate to one another?

School administrator / Developer

DEV Developer

Suppose every term has sensible dates and all dates are in 2026. Can the calendar still be wrong?

SA School administrator

M-2.7 Yes. Term 1 must come before Term 2, and Term 2 before Term 3. They must not overlap.

DEV Developer

If Term 1 closes on 28 March and Term 2 opens on 28 March, is that overlap?

SA School administrator

M-2.8 Yes. The closing day still belongs to Term 1. Term 2 must begin later.

DEV Developer

What if Term 2 is September to November and Term 3 is May to July? They do not overlap.

SA School administrator

Still wrong. The numbering describes their order in the year.

A useful way to state both decisions precisely is to compare an earlier-numbered term with a later-numbered term.

Rule BR-2.6: Earlier-numbered terms must finish before later-numbered terms begin

If two terms are scheduled and one has the lower term number, its closing date must be **strictly before** the later term's opening date.

Business evidence: M-2.7 and M-2.8.

Because boundaries are inclusive, “strictly before” is essential. A Term 1 closing on 28 March and a Term 2 opening on 28 March would share that day and therefore violate BR-2.6.

Example: The boundary between Term 1 and Term 2

Given Term 1 closes on 28 March 2026.

When Term 2 is proposed to open on 28 March and then on 29 March.

Then 28 March is rejected because the terms would share a day; 29 March clears this particular rule.

Example: No overlap, but the numbering is backwards

Given Term 2 is proposed for 1 September–28 November and Term 3 for 1 May–28 July.

When the calendar is checked.

Then it is rejected because Term 2 does not finish before Term 3 begins, even though the two periods do not overlap.

The second example is a **counterexample** to the idea that “non-overlapping” alone is enough.

A rejected change must not become a partial change

One final decision was already present in the Step 1 conversation. If an administrator proposes an invalid reschedule, SchoolGrid should reject the proposal without damaging the dates that were previously accepted.

Rule BR-2.7: Rejecting a change preserves the accepted calendar

If a proposed schedule or reschedule violates a calendar rule, the proposal is rejected and the previously accepted term dates remain unchanged.

For now this is a business requirement, not an implementation technique. Later we will discover what

DISCOVERY TOOL

Counterexample

A concrete case that satisfies a proposed explanation but still produces the wrong business outcome. It shows that the explanation is incomplete.

kind of model makes that guarantee easy to protect.

— The vague requirement is now a decision table

The original request sounded simple: “let the administrator manage term dates.” We can now ask much sharper questions.

Proposal	Reason	Decision
20 May–1 May	Period runs backwards	Reject
20 May–20 May	Does not violate chronology	Not rejected by that rule
Only Term 1 scheduled	Progressive configuration is allowed	Acceptable state
Term 3 closes 15 Jan 2027	Outside Academic Year 2026 policy	Reject
Term 2 opens 28 Mar	Shares Term 1’s closing day	Reject
Term 2 opens 29 Mar	Clears the Term 1 boundary rule	Check remaining rules
Second Term 2 added	Duplicate term number	Reject

Notice the wording in the right-hand column. Passing one rule does not always mean the whole proposal is valid. “29 March clears the Term 1 boundary rule” is more precise than saying “29 March is valid.” Other rules may still matter.

KEY IDEA Business rules become useful when they let us explain both acceptance and rejection with the same vocabulary.

Try it: Challenge the rules with new examples

For each proposal below, identify the rule that decides it. If no single rule is enough, say what else must be checked.

1. Term 1 opens and closes on 10 January 2026.
2. Term 2 is scheduled before Term 1 has been entered.
3. Term 1 closes on 30 April and Term 2 opens on 30 April.
4. Academic Year 2026 contains a term numbered 4.
5. An invalid Term 2 reschedule is submitted after a valid Term 2 already exists.

Do not design classes yet. The goal is to determine what the software must observe, not how the software will be organised.

End-of-step checkpoint

Established Chronology, permitted and unique term numbers, progressive configuration, year containment, ordering, and unchanged state after rejection.

Explain Why does a same-day period clear chronology, while two terms sharing a boundary day fail the ordering rule?

Next evidence Turn an agreed example into a failing test before choosing the implementation.

We now have something stronger than a list of requirements: concrete examples paired with explicit rules. They are still written for humans, however. A future change to the program could violate one of them without anybody noticing.

NEXT QUESTION

How can we turn these agreed examples into executable evidence that guides the implementation and keeps checking it as the code changes?

Next: Step 3 – From Examples to Tests: Introducing TDD

STEP 3

From Examples to Tests: Introducing TDD

What you will discover

Turn an agreed business example into an executable test, explain the red–green–refactor rhythm, and distinguish a useful test failure from a failure that says nothing about the business behaviour.

Step 2 ended with something valuable: the school and the developer could now look at the same proposal and explain why it should be accepted or rejected. For example, BR-2.1 says that a closing date must be on or after its opening date.

That agreement is still written for humans. If somebody changes the program next month, the document will not automatically tell us that the rule has been broken. The next move is therefore small but important: take one of the examples we already trust and make the computer check it for us.

Model evolution

INITIAL INTERPRETATION

Keep the agreed examples in prose.

REVISED UNDERSTANDING

Make one example executable, let it guide a small change, then repeat.

Choose one example, not the whole calendar

We could try to implement every rule from Step 2 immediately. That would hide the reason for each line of code. Instead, we begin with the smallest behaviour we understand well: a period whose closing date comes after its opening date should pass the chronology check.

For the tests in this book we will use **pytest**. Its plain functions and assertions let the examples remain close to the language in which we first discussed them. The first test can be written before the production function exists:

PYTHON

```
1 from datetime import date
2 from schoolgrid.calendar import is_chronological
3
4
5 def test_period_can_end_after_it_starts():
6     start = date(2026, 5, 1)
7     end = date(2026, 7, 28)
8
9     assert is_chronological(start, end)
```

The test says almost exactly what the business example said. It gives concrete input and states the observable result we expect.

DOMAIN WORKSHOP

What should a useful failure tell us?

School administrator / Developer

SA School administrator

If this test fails because you misspelled a file name, have we learned that the calendar rule is wrong?

DEV Developer

No. We have only learned that the test cannot reach the behaviour yet.

SA School administrator

Then what failure would actually teach us something about the rule?

DEV Developer

A failure that reaches the chronology check and shows that the current code accepts or rejects the example incorrectly.

SA School administrator

Good. So “red” is useful only when we understand why it is red.

If we run `pytest -q` now, however, the test cannot even import `is_chronological`. That is a failure, but it is not yet evidence about the chronology rule.

So we add only enough production code for the test to reach its assertion:

PYTHON

```
1 def is_chronological(start, end):
2     return True
```

Now the first example passes. The implementation is almost embarrassingly small. That is acceptable. We have only asked the software one question so far, and `return True` answers that question correctly.

KEY IDEA In TDD, the implementation is allowed to be simpler than the final design. The next example creates the pressure for the next change.

TESTING

Useful failure

A test should fail for the behaviour we are trying to add. A broken import, syntax error, or test setup problem is red, but it tells us nothing about whether the business rule is implemented.

The second example makes the first implementation inadequate

Step 2 also gave us the reversed case: 20 May–1 May must be rejected. We add it as a second test rather than redesigning the function in anticipation.

PYTHON

```
1 def test_period_cannot_end_before_it_starts():
2     start = date(2026, 5, 20)
3     end = date(2026, 5, 1)
```

```

4
5     assert not is_chronological(start, end)

```

This time the test reaches the assertion and fails for the reason we care about: our current implementation returns `True` for a backwards period. This is the kind of failure TDD wants.

The smallest general change that satisfies both examples is now visible:

```

PYTHON
1  def is_chronological(start, end):
2      return end >= start

```

Run the tests again. The normal period still passes, and the reversed period now passes because the function returns `False` for it. We have reached the **green** state.

Notice what did *not* happen. We did not begin by writing a large calendar engine. Two examples forced exactly one comparison into existence.

— A boundary example checks the rule we think we wrote

There was one more chronology example in Step 2: equal opening and closing dates do not run backwards. We add that boundary case explicitly.

```

PYTHON
1  def test_same_day_period_is_chronological():
2      day = date(2026, 5, 20)
3
4      assert is_chronological(day, day)

```

This test passes immediately because `end >= start` already includes equality.

That is not a TDD failure. A new example does not have to force new production code. Sometimes it confirms that the rule we implemented already covers the boundary we intended. What matters is that the behaviour is now executable and will be checked again whenever the suite runs.

Example: The three examples now travel together

Given the test suite contains a normal period, a reversed period, and a same-day period.

When the chronology implementation changes.

Then all three examples are checked again, so a change that fixes one case by breaking another is exposed.

This is why a solution such as “always reject the period” cannot satisfy the suite. It would make the reversed example pass while breaking both accepted examples.

— Red, green, then improve without changing behaviour

We now have enough experience to name the development rhythm.

TDD

Red

An executable example demonstrates that the desired behaviour is missing or wrong. The failure should be relevant to the behaviour being developed.

TDD

Green

Make the current examples pass with the simplest clear implementation that satisfies them. Green is not permission to ignore design; it gives us a safe point from which to improve it.

TDD

Refactor

Improve the structure, names, or duplication of working code while keeping its externally observed behaviour unchanged. The tests should remain green throughout.

Move	What we do
Red	Add one executable example that exposes missing or incorrect behaviour.
Green	Make that example and all earlier examples pass with a small implementation change.
Refactor	Improve working code without changing the behaviour the tests observe.

There is no useful refactoring to perform on our one-line function yet. That is also a valid decision. **Refactor is an opportunity, not a ceremony.** We should not make the design more elaborate merely because the cycle contains a word called refactor.

The important part is the order:

1. choose the next behaviour we understand;
2. express it as an executable example;
3. see the relevant failure;
4. make the smallest clear change that satisfies it;
5. improve the design only when there is something worth improving.

— This is different from testing everything afterwards

Suppose we first build the entire calendar subsystem and only then write tests for all the rules from Step 2. Those tests may be useful, but they did not guide the design as it grew.

TDD uses tests earlier. The next example participates in deciding what code needs to exist. The implementation and the examples grow together.

CONTRAST

Testing after implementation

Code is designed first; tests later check what was built.

Test-driven development

One desired behaviour is made executable before the implementation for that behaviour is completed.

TDD also does *not* mean writing every conceivable test for the whole system before writing any production code. We are doing the opposite: one small example, one small implementation step, then the next example.

— A red test can still be the wrong red

Imagine we accidentally type this import:

PYTHON

```
1 from schoolgrid.calender import is_chronological
```

The test run fails because `calender` is misspelled. The screen is red, but the failure tells us nothing about whether a reversed period should be accepted.

Before changing production logic, ask a simple diagnostic question:

Is the test failing because the expected business behaviour is absent, or because the test cannot reach that behaviour at all?

This distinction becomes increasingly important as systems gain databases, frameworks, networks, and external services. A failing test is information; we still have to interpret what information it contains.

Tests should describe behaviour, not our current implementation

Our tests call `is_chronological` and assert its result. They do not check which operator the function uses internally, how many local variables it creates, or what line of code performs the comparison.

That is deliberate. A useful test gives us freedom to improve the implementation while preserving the observable rule. If we later replace this crude function with a richer model, the business examples should still make sense.

Example: A safe internal change

Given all chronology examples are green.

When the internal implementation is reorganised without changing which periods are accepted or rejected.

Then the tests should remain green because the observed behaviour has not changed.

This is the protection we wanted at the end of Step 2: the rules no longer depend only on somebody remembering to reread the document after every code change.

The remaining rules become a development queue

We have automated only BR-2.1. The other agreements from Step 2 are still waiting. We do not need to implement them all in this step, but we can now see how they will enter development.

Business example	Possible next executable behaviour
Term number 4 is proposed	Reject term numbers outside 1, 2, and 3.
A second Term 2 is added	Reject a duplicate term number.
Only Term 1 is scheduled	Allow a partially configured academic year.
Term 3 closes in January 2027	Reject a term outside the 2026 calendar-year policy.
Term 2 opens on Term 1's closing day	Reject terms that share a day.
An invalid reschedule is submitted	Preserve the previously accepted dates.

Each row can become a test when we are ready to implement that behaviour. The examples give us a disciplined order of work rather than a reason to design the whole system in advance.

Try it: Choose the next red test

Assume the three chronology tests are green. Choose one rule from the table above and write, in plain language, the *single* example you would automate next. Then answer these questions:

1. What observable result should the test assert?
2. What would count as a meaningful failure?
3. What kind of failure would merely indicate broken test setup?
4. What is the smallest production behaviour that the example requires?

Do not design the final class structure yet.

Our tests are already beginning to contain the vocabulary of the school: **term**, **opening date**, **closing date**, **academic year**, **reschedule**. If those words mean one thing to the administrator and another to the developer, executable tests will only make the misunderstanding faster.

NEXT QUESTION

Before the model grows, which words belong to this problem, who defines their meaning, and how do we make the same language appear in conversation, tests, and code?

Next: Step 4 – Domain, Domain Experts, and Ubiquitous Language

STEP 4

Domain, Domain Experts, and Ubiquitous Language

What you will discover

Explain what a domain and a domain expert are, build a shared language from a real misunderstanding, and make that language visible in conversation, tests, and small pieces of code.

Step 3 left us with executable examples. The tests can now tell us when a chronology rule is broken. But the last lines of that step exposed a different risk: the software is beginning to contain words such as **term**, **opening date**, **academic year**, and **reschedule**.

Tests cannot protect us from a word that the team understands in two different ways. They can faithfully automate the wrong meaning.

The first problem is not code

The developer is preparing the next test from the Step 2 rule that prohibits a second Term 2. Before writing it, he describes what he plans to implement.

DOMAIN WORKSHOP

What exactly are we changing?

School administrator / Developer

DEV Developer

Next I will test that the system cannot create two Semester 2 records for the same year.

SA School administrator

We do not call them semesters here. They are terms.

DEV Developer

Fine—two Term 2 records. If the dates change, I suppose I delete the old record and create another Term 2?

SA School administrator

No. That sounds like a different term. The headteacher is changing the dates of the existing Term 2. We call that rescheduling it.

DEV Developer

So “schedule Term 2” and “reschedule Term 2” are different actions?



School administrator

Yes. Scheduling gives a term its period. Rescheduling changes the period of a term that is already scheduled.

Nothing has failed in Python. Yet the conversation has exposed three possible misunderstandings: *semester* versus *term*, *creating* versus *changing*, and *schedule* versus *reschedule*.

Model evolution

INITIAL INTERPRETATION

Assume ordinary words are close enough.

REVISED UNDERSTANDING

Agree on the business meaning of important words, then use those meanings consistently.

The work itself gives the words their meaning

Why should the administrator's correction matter? Because SchoolGrid is being built for a particular area of work: planning and maintaining the school's academic calendar.

In Domain-Driven Design, that area of knowledge and activity is the **domain**.

The domain is not the database, the Python package, or the user interface. Those are parts of a software solution. The domain exists even if the school is still using paper, spreadsheets, or meetings to manage it.

Our school has many other areas of work: admissions, fees, examinations, attendance, staffing, and timetabling. We are not trying to model all of them now. The domain knowledge relevant to this part of SchoolGrid is deliberately narrower: **academic-calendar planning**.

IN THIS SLICE

Academic years, terms, term numbers, periods, scheduling, rescheduling, and the rules that make a calendar acceptable.

OUTSIDE FOR NOW

Fees, attendance, student enrolment, lesson timetables, examinations, staffing, and other school operations.

This boundary also affects language. A word may have a useful meaning in one area of work and a different meaning elsewhere. We therefore ask, "What does this word mean *for this problem*?" rather than trying to invent a universal definition for the whole school.

Who can correct the software team's assumptions?

The administrator did more than supply dates. She knew that 28 March still belongs to Term 1, that a year may be configured progressively, and that changing an existing term's dates is a reschedule rather than a new term.

That kind of knowledge makes her a **domain expert** for the calendar work we are discussing.

A domain expert does not have to be a programmer, a manager, or the person with the most senior job title. Expertise is contextual. The bursar may be the better expert for school fees; an examinations officer may be the better expert for assessment scheduling.

Likewise, the developer contributes a different kind of expertise. He can expose ambiguity, construct counterexamples, and show where a vague sentence would force the software to guess. Good mod-

DDD CONCEPT

Domain

The area of knowledge, activity, rules, and problems that the software is intended to support. Here, our current focus is academic-calendar planning.

DDD CONCEPT

Domain expert

A person with authoritative or practical knowledge of how the domain works, including its terminology, decisions, exceptions, and business rules.

elling comes from putting these forms of knowledge into the same conversation.

CONTRAST

Domain expertise

Knowledge of the school's calendar work: meanings, rules, decisions, exceptions, and consequences.

Software expertise

Knowledge of how to make those decisions explicit, executable, testable, and maintainable in software.

Neither side can safely replace the other.

Build the vocabulary from decisions we already made

Instead of starting with a dictionary, the developer returns to the concrete rules and asks what each important word must mean for those rules to make sense.

DOMAIN WORKSHOP

Agreeing on the calendar vocabulary

School administrator / Developer

DEV Developer

When you say "Academic Year 2026," what are you referring to?

SA School administrator

The school year whose calendar we are configuring for 2026. It contains the terms we schedule for that year.

DEV Developer

And a term?

SA School administrator

One numbered teaching period in that academic year—Term 1, Term 2, or Term 3.

DEV Developer

You just used "period." Is that another word for term?

SA School administrator

No. The term is Term 2. Its period is the span from its opening date to its closing date. If we move those dates, it is still Term 2.

DEV Developer

Then the term number tells us which term it is and where it belongs in the sequence?

SA School administrator

Exactly.

That conversation gives us a first working vocabulary.

Business term	Meaning in this context
Academic Year	The school year whose academic calendar is being configured, such as Academic Year 2026.
Term	A numbered teaching period within an academic year: Term 1, Term 2, or Term 3 in this scenario.
Term number	The numbered position of a term in the academic year: 1, 2, or 3.
Period	The inclusive span from a term's opening date through its closing date.
Schedule	Give a term a period when it is not yet scheduled.
Reschedule	Change the period of a term that is already scheduled.
Opening date	The first day included in a period.
Closing date	The final day included in a period.

These are *working* definitions. If further conversations reveal that one of them is misleading, the team should change the language and change the code that expresses it. Shared language is discovered and refined; it is not frozen because somebody once typed a glossary.

— The distinction between a term and its period matters

The word choice now changes how we interpret an earlier request.

Example: Reschedule Term 2

Given Term 2 already exists with a period from 1 May to 28 July 2026.

When the headteacher asks to reschedule Term 2 to open on 15 April.

Then the request means changing the period of the existing Term 2, not creating another Term 2.

If we treated *term* and *period* as synonyms, a date change could look like replacement of the whole term. But the administrator has told us something more precise: **the term remains Term 2 while its dates may change**.

We will not yet decide what kind of DDD object a Term should become. That is a modelling question for later. At this point we only need to preserve the business meaning instead of losing it behind generic words such as “record.”

— Now the tests can speak more like the school

Our first TDD examples used short technical names:

PYTHON

```
1 def test_period_cannot_end_before_it_starts():
2     start = date(2026, 5, 20)
3     end = date(2026, 5, 1)
4
5     assert not is_chronological(start, end)
```

The behaviour is correct, but `start` and `end` throw away some of the vocabulary we have just agreed. We can improve the test without changing its behaviour:

PYTHON

```

1 def test_period_cannot_close_before_it_opens():
2     opening_date = date(2026, 5, 20)
3     closing_date = date(2026, 5, 1)
4
5     assert not is_chronological(opening_date, closing_date)

```

The production function can adopt the same parameter names:

PYTHON

```

1 def is_chronological(opening_date, closing_date):
2     return closing_date >= opening_date

```

No algorithm changed. This is a refactoring of language. The test and the code now preserve more of what the school actually says.

KEY IDEA Names in code are modelling decisions. When business language is precise, carrying that language into tests and code reduces translation and exposes misunderstandings earlier.

Shared language is more than a glossary

DDD calls this deliberately shared vocabulary a **Ubiquitous Language**.

“Ubiquitous” does not mean that every person in the world must use these words. It means the language should be pervasive *within the collaboration and model for this domain*. If the administrator says *reschedule*, the requirement says *reschedule*, but the code says `replace_calendar_row`, the team is continually translating between two languages.

A glossary can help record meanings, but the language becomes valuable only when we actually use it:

Where	Example of the shared language
Conversation	“Reschedule Term 2.”
Business rule	“A term number occurs at most once within an academic year.”
Example	“Term 2 opens on Term 1’s closing day.”
Test	<code>test_period_cannot_close_before_it_opens</code>
Code	<code>opening_date, closing_date</code>

The same ideas travel from the school conversation into executable software with less translation at every boundary.

Bad language often hides a missing decision

Suppose a developer says, “I will update the calendar record.” That sentence is technically plausible but weak as domain language. Which record? What business change is taking place? Are we scheduling a new term, rescheduling an existing term, or changing the academic year itself?

Generic technical words are not forbidden. Databases really do have rows and records. But those words should not replace meaningful domain language when we are describing domain behaviour.

DDD CONCEPT

Ubiquitous Language

A shared, evolving language developed by domain experts and software practitioners and used consistently in conversations, rules, examples, tests, and the model.

CONTRAST**Technical description**

“Update the Term 2 record’s start field.”

Domain description

“Reschedule Term 2 with an earlier opening date.”

The second description exposes more questions immediately: Does Term 2 already exist? Is the new period acceptable? What happens if the reschedule is rejected? Those are exactly the decisions SchoolGrid must protect.

— The language also tells us what not to assume

A shared term is useful only when its meaning is explicit. Consider **schedule**. The administrator defined it as assigning a period to a term that is not yet scheduled. That means these two statements are not interchangeable:

Example: Schedule

Given Term 2 has no period yet.

When the administrator assigns it an opening and closing date.

Then Term 2 is being scheduled.

Example: Reschedule

Given Term 2 already has an accepted period.

When the administrator proposes different opening or closing dates.

Then Term 2 is being rescheduled, and a rejected proposal must leave the accepted period unchanged.

The difference is not decorative vocabulary. The second operation has an existing accepted state to preserve; the first does not.

— A language check before we add more code

We can now revisit the examples waiting in the TDD queue and phrase them using the shared vocabulary:

Weak phrasing	Domain phrasing
Add number 4	Schedule a term with term number 4.
Duplicate row 2	Schedule a second Term 2 in the same academic year.
Change start field	Reschedule Term 2 with a new opening date.
Dates cross years	Schedule a term whose period extends outside Academic Year 2026.
Two records touch	Schedule Term 2 to open on Term 1’s closing date.

The right-hand column gives us better candidates for future test names because it states what happens in the school’s world rather than how we happen to store it.

Try it: Repair the language before repairing the code

For each phrase below, identify what is ambiguous or overly technical, then rewrite it using the vocabulary established in this step:

1. "Insert semester 2."
2. "Change the start field on record 2."
3. "Create another Term 2 with the corrected dates."
4. "The two date records collide."

For each rewrite, state one business question that becomes easier to ask once the language is more precise.

We now have rules, executable examples, and a vocabulary that connects the school conversation to the code. But our implementation still passes loose pairs of dates around. Every caller must remember what those dates mean and which rules belong with them.

NEXT QUESTION

If the language says that periods, terms, and academic years are meaningful concepts with different responsibilities, how should those concepts begin to appear in the software model?

Next: Step 5 – From Business Rules to a Domain Model

STEP 5

From Business Rules to a Domain Model

What you will discover

Use business rules to discover candidate domain concepts and responsibilities, distinguish rules that can be decided locally from rules that require wider context, and build a provisional School-Grid model without prematurely naming DDD patterns.

Step 4 gave the team a much better language. We can now say that a **Term** has a **period**, that its **term number** places it in the academic year, and that **rescheduling** changes the period of an existing term.

The code has not caught up with that understanding. Our chronology function still receives two loose values:

PYTHON

```
1 def is_chronological(opening_date, closing_date):
2     return closing_date >= opening_date
```

That was enough for one rule. The next rules make the weakness harder to ignore.

The same pair of dates starts travelling everywhere

To implement more of the Step 2 rules with only primitive values, we might begin adding functions like these:

PYTHON

```
1 def is_chronological(opening_date, closing_date):
2     ...
3
4
5 def is_inside_year(year, opening_date, closing_date):
6     ...
7
8
9 def schedule_term(year, term_number, opening_date, closing_date):
10    ...
```

The opening and closing dates keep travelling together, but Python does not know that they belong together. Every function must remember which argument is which and which relationships matter.

The problem becomes more obvious at a call site:

PYTHON

```
1 schedule_term(
2     2026,
3     2,
```

```

4     date(2026, 4, 15),
5     date(2026, 7, 28),
6 )

```

A reader has to reconstruct the meaning of four separate arguments. Worse, these values can be rearranged accidentally without Python objecting.

DOMAIN WORKSHOP

Do these dates mean something together?

School administrator / Developer

DEV Developer

I keep passing an opening date and a closing date together. From the school's point of view, are they two unrelated facts?

SA School administrator

No. Together they describe the term's period.

DEV Developer

So when I check whether the closing date is before the opening date, I am really asking whether that period makes sense?

SA School administrator

Yes. That decision belongs to the period itself. You should not need to rediscover it every place that uses the dates.

The conversation has created a modelling pressure: the software understands two separate dates, while the school understands **one meaningful span**.

Model evolution

INITIAL INTERPRETATION

Pass opening and closing dates as unrelated arguments.

REVISED UNDERSTANDING

Represent the date span once, then let the rules tell us what responsibility belongs with it.

— A domain model is a useful representation, not a class inventory

We now have enough pressure to introduce another DDD idea. A **domain model** is a purposeful representation of the concepts, relationships, and behaviour needed to solve the domain problem.

The word *selective* matters. SchoolGrid does not need a Python class for Headteacher merely because a headteacher appeared in a conversation. Nor does it need a class called OpeningDate merely because that phrase is important.

A better question is:

Which concept has enough information to make this business decision, and where would putting the rule make the model easier to understand and protect?

DDD CONCEPT

Domain model

A selective representation of domain concepts and rules used by the software to reason about the business problem. It is not a copy of every real-world noun or every database table.

We can apply that question directly to the rules already discovered.

— The first responsibility needs only one span of dates

Consider BR-2.1: a closing date must be on or after its opening date.

To decide that rule, we need exactly two pieces of information:

Information needed	Information not needed
Opening date	Term number
Closing date	Other terms
	Academic year schedule

The rule can therefore be decided by the concept that represents the date span. The school calls that span a *period*. In code, the team chooses the name `DateRange` for the object that represents the pair of inclusive boundary dates. A Term's period will be represented by one of these date ranges.

That is a refinement of the shared language rather than a replacement for it: *period* remains the business idea, while `DateRange` makes the structure of that period explicit in Python.

Our first sketch is deliberately small:

PYTHON

```
1 from dataclasses import dataclass
2 from datetime import date
3
4
5 @dataclass
6 class DateRange:
7     opening_date: date
8     closing_date: date
```

Now one value carries the two dates together:

PYTHON

```
1 term_2_period = DateRange(
2     opening_date=date(2026, 5, 1),
3     closing_date=date(2026, 7, 28),
4 )
```

This alone does not enforce any rule. It simply gives the date pair a meaningful home.

— Put a rule where all of its information already lives

The next question is whether callers should still have to unpack the range and call a separate chronology function.

They should not. The `DateRange` already contains everything required to answer the chronology question, so the behaviour can move next to the data it uses:

PYTHON

```
1 @dataclass
2 class DateRange:
3     opening_date: date
```

```

4     closing_date: date
5
6     def is_chronological(self):
7         return self.closing_date >= self.opening_date

```

The corresponding test becomes:

```

PYTHON
1  def test_date_range_cannot_close_before_it_opens():
2      period = DateRange(
3          date(2026, 5, 20),
4          date(2026, 5, 1),
5      )
6
7      assert not period.is_chronological()

```

The algorithm has not become more sophisticated. The improvement is about **responsibility**: code that has a `DateRange` can ask the `DateRange` about its own chronology instead of reconstructing the rule elsewhere.

Model decision: Chronology belongs with the date range

The relationship between an opening date and a closing date can be decided using that date range alone. The model should therefore not require an `AcademicYear`, a `Term`, or an application service merely to answer whether the boundaries run forwards.

There is still an obvious weakness: we can construct a backwards `DateRange` and only discover the problem when somebody remembers to call `is_chronological()`. We will return to that pressure soon. For now, the important discovery is where the knowledge belongs.

— A term carries different information

The next rules cannot be answered by a `DateRange` alone. A date range such as 1 May–28 July does not tell us whether it belongs to Term 1, Term 2, or Term 3.

The domain language from Step 4 tells us that a **Term** has a numbered position and may have a period. Because the academic year may be configured progressively, a term may exist before its dates are assigned.

A provisional representation could therefore look like this:

```

PYTHON
1  @dataclass
2  class Term:
3      number: int
4      period: DateRange | None = None

```

This code is intentionally incomplete. An ordinary `int` would allow 4, 99, or `-1`, even though BR-2.2 permits only 1, 2, and 3. We are not fixing every weakness as soon as we notice it. We are letting each weakness create the next modelling question.

What has improved is the shape of the software:

PYTHON

```

1 term_2 = Term(
2     number=2,
3     period=DateRange(
4         date(2026, 5, 1),
5         date(2026, 7, 28),
6     ),
7 )

```

The code now says something recognisable in the school's language: Term 2 has a period.

Some rules require more than one term

Now consider BR-2.3. Can one Term decide whether it is a duplicate?

No. A Term can know that its number is 2, but it cannot know whether another Term 2 is already scheduled unless it can see the other terms in the same academic year.

The same problem appears with BR-2.6. A DateRange can know its own boundaries. A Term can know its own number and period. Neither can decide whether Term 1 finishes before Term 2 begins without seeing both terms.

DOMAIN WORKSHOP

Who knows whether the calendar fits together?

School administrator / Developer

DEV Developer

Can Term 2 decide by itself whether it overlaps Term 1?

SA School administrator

Not without knowing Term 1's dates.

DEV Developer

And can Term 2 decide whether another Term 2 already exists?

SA School administrator

Again, it needs the rest of the year's schedule.

DEV Developer

Then those are not really questions about one term. They are questions about the academic year containing the terms.

SA School administrator

That matches how we work. We judge whether the year's calendar fits together.

The conversation gives us a third candidate concept:

PYTHON

```

1  @dataclass
2  class AcademicYear:
3      year: int
4      terms: list[Term]

```

Again, this is only a sketch. It does not yet protect the list from duplicates, overlaps, or arbitrary mutation. Those are later design problems. The model is starting by making the relevant concepts and relationships visible.

— Use the rules to assign provisional responsibilities

We can now revisit each business rule and ask what information is required to decide it.

Rule or question	Information needed	Provisional home
Closing date is not before opening date	One date range	DateRange
Is this Term 1, 2, or 3?	One term number	A more meaningful representation of the term number is likely needed
What period is assigned to Term 2?	One term	Term
Does another Term 2 already exist?	Several terms in one year	AcademicYear
Do two scheduled terms share a day?	Several term periods	AcademicYear
Does term chronology follow numbering?	Several terms and their numbers	AcademicYear
Do scheduled dates belong to 2026?	Year plus term period	AcademicYear
May only Term 1 be scheduled for now?	The year's current schedule	AcademicYear

The right-hand column is *provisional*. We are not proving a final class design. We are using the information required by each decision to avoid placing rules arbitrarily.

KEY IDEA A useful modelling heuristic is to ask which concept has enough information to decide the rule. Local rules should not require the whole system; coordinating rules should not be pushed into an object that cannot see the necessary context.

— Local validity is not calendar validity

The distinction becomes concrete if we construct two individually sensible date ranges:

PYTHON

```

1  term_1_period = DateRange(
2      date(2026, 1, 1),
3      date(2026, 3, 28),
4  )
5
6  term_2_period = DateRange(
7      date(2026, 3, 28),
8      date(2026, 7, 28),
9  )

```

Each range is chronological. Yet together they violate the calendar rule because both include 28 March.

Example: Valid parts, invalid combination

Given Term 1 has a chronological period ending on 28 March and Term 2 has a chronological period opening on 28 March.

When the two terms are considered in the same academic year.

Then each period passes its local chronology rule, but the academic-year schedule is rejected because the terms share a day.

This is a crucial modelling distinction. A concept can be locally valid without the larger combination being valid.

If we put the overlap rule inside `DateRange`, we would have to give one range knowledge of other ranges that do not belong to it. If we put chronology only in `AcademicYear`, we would force the whole calendar to answer a question that one range can already answer by itself.

Good responsibility assignment keeps both mistakes out of the model.

Do not turn every noun into a class

At this point it would be easy to become enthusiastic and create classes for `OpeningDate`, `ClosingDate`, `Headteacher`, `Administrator`, `ScheduleRequest`, and every other noun we have used.

That is not how a domain model is discovered.

A concept earns a place in the model because it carries meaning, behaviour, constraints, or relationships that matter to the problem. Some nouns remain ordinary values. Some people remain actors outside the model. Some concepts will become important only when later requirements create pressure for them.

CONTRAST

Noun-driven modelling

Find every noun in the requirements and make it a class.

Responsibility-driven modelling

Use rules and decisions to discover which concepts need to carry meaning or protect behaviour.

For `SchoolGrid`, `DateRange`, `Term`, and `AcademicYear` have appeared because the rules repeatedly require those concepts to answer different kinds of questions.

The first model is allowed to be imperfect

Our sketch currently looks like this:

PYTHON

```
1 @dataclass
2 class DateRange:
3     opening_date: date
4     closing_date: date
5
6     def is_chronological(self):
7         return self.closing_date >= self.opening_date
```

```

8
9
10 @dataclass
11 class Term:
12     number: int
13     period: DateRange | None = None
14
15
16 @dataclass
17 class AcademicYear:
18     year: int
19     terms: list[Term]

```

This is not the final SchoolGrid model. In fact, the examples already expose several weaknesses:

- a backwards DateRange can still be constructed;
- a Term can still receive number 4;
- callers could insert duplicate terms directly into the list;
- nothing yet protects term ordering or overlap;
- nothing yet ensures a rejected reschedule preserves the accepted state.

Those weaknesses are useful. They tell us where the next design pressure lives. We should resist the temptation to solve all of them in one giant refactor.

TDD gives us a safer path: choose one behaviour, make the problem executable, and let the model become stronger in response.

— A new question appears inside the smallest concept

The DateRange is the simplest part of the sketch, so we inspect it first.

Two administrators independently enter the same period:

```

PYTHON
1 first = DateRange(
2     date(2026, 5, 1),
3     date(2026, 7, 28),
4 )
5
6 second = DateRange(
7     date(2026, 5, 1),
8     date(2026, 7, 28),
9 )

```

They are two Python objects created at different moments. But in the school's language, do they describe two different periods, or the same date span expressed twice?

That question is no longer about where to store a rule. It is about what kind of thing a DateRange *is*: whether its meaning comes from a separate identity or from the values it contains.

Try it: Find the smallest concept that can decide

For each question below, identify the minimum information required and the most reasonable provisional home in the model. Explain why a smaller concept cannot decide it and why a larger concept would be unnecessary.

1. Is 20 May–1 May a sensible date span?
 2. Is term number 4 allowed?
 3. Is there already a Term 2 in Academic Year 2026?
 4. Do Term 1 and Term 2 share a calendar day?
 5. If a proposed Term 2 reschedule is rejected, which accepted state must remain unchanged?
- Do not name DDD tactical patterns yet. Focus on information and responsibility.

NEXT QUESTION

If two independently created `DateRange` objects contain the same opening and closing dates, should the domain treat them as different things or as the same value?

Next: Step 6 – Value Objects: Modelling Meaningful Values

STEP 6

Value Objects: Modelling Meaningful Values

What you will discover

Explain why two independently created date ranges can represent the same domain value, distinguish value equality from object identity, and recognise why a `DateRange` does not need a separate business identifier.

Step 5 ended with two administrators independently describing exactly the same period: 1 May through 28 July 2026. In Python, those descriptions can become two separate objects:

PYTHON

```
1 first = DateRange(
2     date(2026, 5, 1),
3     date(2026, 7, 28),
4 )
5
6 second = DateRange(
7     date(2026, 5, 1),
8     date(2026, 7, 28),
9 )
```

The objects were created separately. The business question is whether that fact matters.

Ask the school before asking Python

The developer takes the question back to the administrator.

DOMAIN WORKSHOP

Are these two different periods?

School administrator / Developer

DEV Developer

Suppose you enter 1 May to 28 July and another administrator independently enters 1 May to 28 July. Do those describe two different periods?

SA School administrator

No. They describe the same span of dates.

DEV Developer

Even though the entries were made by different people at different times?

SA School administrator

That does not change the period. If the opening and closing dates are the same, the period is the same.

DEV Developer

Would you ever need to ask which one of those two periods is the original one?

SA School administrator

No. For a period, the dates are what matter.

That last answer changes how we should model the concept. The school does not care about the history of the Python object. It cares about the **values that describe the period**.

Model evolution**INITIAL INTERPRETATION**

Treat every object instance as a different thing.

REVISED UNDERSTANDING

Ask whether the domain distinguishes instances by identity or only by the values they contain.

Python has two different questions

Python lets us make the distinction visible. With the provisional dataclass from Step 5, these two expressions answer different questions:

PYTHON

```
1 first is second    # False
2 first == second   # True
```

`is` asks whether both variables refer to the **same Python object**. They do not. Two constructors ran, so two objects exist in memory.

`==` asks whether the objects are **equal according to their value**. Python dataclasses generate equality from their fields by default, so the two `DateRanges` compare equal because both boundary dates match.

The administrator's answer tells us that `==` is the comparison that matches the domain meaning.

Example: Two constructions, one domain value

Given two `DateRange` objects are constructed independently with opening date 1 May 2026 and closing date 28 July 2026.

When their meaning in the school calendar is compared.

Then they represent the same period even though they are not the same Python object.

This gives us an executable example worth keeping:

PYTHON

```
1 def test_date_ranges_with_same_boundaries_are_equal():
2     first = DateRange(
3         date(2026, 5, 1),
```

```

4         date(2026, 7, 28),
5     )
6     second = DateRange(
7         date(2026, 5, 1),
8         date(2026, 7, 28),
9     )
10
11     assert first == second

```

The test may already pass because `@dataclass` gave us field-based equality in Step 5. That is useful information rather than a problem. We have just discovered that an existing Python behaviour happens to match a newly explicit business expectation.

TDD does not require us to damage the implementation just to manufacture a red test. The important step is to record the example so future changes must preserve its meaning.

— Change one value and the meaning changes

The administrator's rule also tells us what should make two periods different. If one boundary changes, the date span changes.

PYTHON

```

1  def test_different_boundary_means_different_date_range():
2      original = DateRange(
3          date(2026, 5, 1),
4          date(2026, 7, 28),
5      )
6      changed = DateRange(
7          date(2026, 4, 15),
8          date(2026, 7, 28),
9      )
10
11     assert original != changed

```

The two ranges differ only in the opening date, but that is enough. The values that define the period are no longer the same.

CONTRAST

Same value

1 May–28 July compared with another independently constructed 1 May–28 July.

Different value

1 May–28 July compared with 15 April–28 July.

The distinction is about meaning, not allocation in memory.

— Now the DDD concept has earned a name

We have discovered a concept whose meaning is completely described by its attributes and for which separate identity has no business significance. Domain-Driven Design calls this kind of concept a **Value Object**.

DDD CONCEPT

Value Object

A domain concept whose equality is based on the values that describe it rather than on a separate continuing identity.

DDD discovery: Value Object

For SchoolGrid, a `DateRange` is a Value Object because its domain meaning is the combination of its opening and closing dates. Two `DateRanges` with equal boundaries represent the same value, regardless of when or where the Python objects were created.

A Value Object is not simply “an object that contains values.” The important question is how the *domain* distinguishes one instance from another.

If changing the descriptive values means we now have a different value, and the business has no independent notion of “which exact one” must be tracked through time, value semantics are a strong fit.

Do we need an identifier anyway?

Developers are accustomed to adding IDs. It is tempting to give `DateRange` its own identifier immediately:

PYTHON

```
1 @dataclass
2 class DateRange:
3     range_id: str
4     opening_date: date
5     closing_date: date
```

Now imagine two administrators enter the same period but the system generates two identifiers:

PYTHON

```
1 first = DateRange(
2     "range-17",
3     date(2026, 5, 1),
4     date(2026, 7, 28),
5 )
6
7 second = DateRange(
8     "range-42",
9     date(2026, 5, 1),
10    date(2026, 7, 28),
11 )
```

If the identifier participates in equality, these objects now compare as different even though the administrator just told us they describe the same period.

The right question is therefore not “Can we add an ID?” We obviously can. The question is:

What business question would this identifier answer that the opening and closing dates cannot answer?

For `DateRange`, we currently have no such question.

Model decision: `DateRange` has value equality, not business identity

SchoolGrid compares `DateRanges` using their boundary dates. We do not introduce a separate domain identifier merely because a `DateRange` is represented by an object or might eventually

be stored somewhere.

A database may later use a technical key for storage mechanics. That does not mean the key must become part of the domain concept or define its equality. We will keep persistence concerns separate from business meaning.

— The term exposes a different kind of question

The distinction becomes clearer if we compare `DateRange` with the `Term` that uses it.

In Step 4, the administrator said that rescheduling Term 2 changes its period but it is still **Term 2**.

Suppose the original period is:

PYTHON

```
1 original_period = DateRange(  
2     date(2026, 5, 1),  
3     date(2026, 7, 28),  
4 )
```

and the accepted reschedule uses:

PYTHON

```
1 new_period = DateRange(  
2     date(2026, 4, 15),  
3     date(2026, 7, 28),  
4 )
```

Those are two different `DateRange` values. The opening date changed, so the value changed.

But the school still speaks about *Term 2 before the reschedule* and *Term 2 after the reschedule*. Something about Term 2 continues even while one of its descriptive values changes.

We will not name that second DDD pattern yet. For now, it is enough to notice that **not every domain concept should use the same equality rule**.

Example: A new period does not imply a new term

Given Term 2 originally has period 1 May–28 July 2026.

When the school accepts a reschedule to 15 April–28 July 2026.

Then the `DateRange` value is different, while the business still refers to the continuing Term 2.

This is why “make every domain class a Value Object” would be just as misguided as “give every domain class an ID.” The domain decides which distinction matters.

— Value equality makes tests read naturally

Once equality reflects domain meaning, tests can compare expected and actual values directly.

Imagine some code calculates the period that should be proposed for Term 2:

PYTHON

```

1 expected = DateRange(
2     date(2026, 4, 15),
3     date(2026, 7, 28),
4 )
5
6 actual = proposed_period_for_term_2()
7
8 assert actual == expected

```

The assertion says what the test means: the returned period should have the same value as the expected period. It does not need to inspect object addresses or invent identifiers simply to compare two date spans.

This property becomes especially useful as models grow. Small meaningful values can travel through the model while retaining their own equality semantics.

But our DateRange is still too easy to damage

We have improved the meaning of equality, but inspect the current class again:

PYTHON

```

1 @dataclass
2 class DateRange:
3     opening_date: date
4     closing_date: date
5
6     def is_chronological(self):
7         return self.closing_date >= self.opening_date

```

Two weaknesses from Step 5 remain.

First, a backwards range can still be created:

PYTHON

```

1 broken = DateRange(
2     date(2026, 5, 20),
3     date(2026, 5, 1),
4 )

```

The object exists even though it represents a date span that violates BR-2.1. We only discover the problem if somebody remembers to call `is_chronological()`.

Second, its boundaries can currently be reassigned after construction:

PYTHON

```

1 period = DateRange(
2     date(2026, 5, 1),
3     date(2026, 7, 28),
4 )
5
6 period.closing_date = date(2026, 4, 1)

```

The same Python object has now changed from one value into another—and possibly into an invalid one. That sits awkwardly with the idea we just discovered: a `DateRange` is meaningful because of the

values that describe it.

These are not equality problems. They are questions about how a Value Object should protect its own meaning throughout its life.

KEY IDEA Value Object is primarily a statement about domain meaning and equality. Once that meaning is clear, it creates further design questions about invariants, behaviour, and mutation.

— A precise mental model before moving on

For the `DateRange` we have learned four things so far:

Question	SchoolGrid answer
What describes it?	Opening date and closing date.
When are two equal?	When both boundary dates are equal.
Does separate object identity matter?	No. Independently created equal ranges represent the same domain value.
Does it need a business ID?	No business requirement discovered so far requires one.

Notice what is *not* in the table yet: how creation should reject invalid boundaries, whether the value should be immutable, how many days it contains, whether a date lies inside it, or whether two ranges overlap. Those are behaviour and invariant questions created by the same concept, and they deserve their own step.

Try it: Identity or value?

For each description below, decide what evidence you would need before modelling it by value. Do not classify by noun alone.

1. Two independently created `DateRanges` both represent 1 May–28 July 2026.
2. Term 2 changes its period after a reschedule.
3. Two strings both contain the text “Term 2.”
4. Two proposed periods have the same dates but were submitted by different administrators.

For each case, ask: if all descriptive values are equal, does the business still need to distinguish *which one* it is? Explain your answer using the domain, not Python object addresses or database keys.

NEXT QUESTION

If a `DateRange` is defined by its values, should those values be allowed to mutate or become invalid after construction—and what useful behaviour should the range itself provide?

Next: Step 7 – Value Object Behaviour and Invariants

STEP 7

Value Object Behaviour and Invariants

What you will discover

Protect a Value Object's validity throughout its life, explain why immutability fits value semantics, and derive inclusive duration, membership, and overlap behaviour from concrete calendar questions.

Step 6 established what a `DateRange` *is*: its meaning comes from its opening and closing dates, and two ranges with equal boundaries represent the same domain value.

But the class still permits two uncomfortable things. We can create a backwards range, and we can mutate a valid range into a different—possibly invalid—one:

PYTHON

```
1 period = DateRange(
2     date(2026, 5, 1),
3     date(2026, 7, 28),
4 )
5
6 period.closing_date = date(2026, 4, 1)
```

The equality semantics are right, but the object does not yet protect the meaning we gave it.

A backwards `DateRange` should never become an accepted value

Our current design contains this method:

PYTHON

```
1 def is_chronological(self):
2     return self.closing_date >= self.opening_date
```

That method can detect a problem, but only after the problematic object already exists. Every caller must remember to ask the question.

The developer takes that weakness back to the school conversation.

DOMAIN WORKSHOP

Should an invalid period exist first and be checked later?

School administrator / Developer



Developer

Right now I can create a period from 20 May to 1 May and then ask whether it is chronological.

SA School administrator

Why would that ever count as a period we can use? It closes before it opens.

DEV Developer

So rather than creating it and hoping every caller checks it, construction itself should fail?

SA School administrator

Yes. If those boundaries do not describe a valid period, do not let that period into the calendar work.

This gives us a stronger expectation than the predicate from Step 3. We can make that expectation executable:

PYTHON

```
1 def test_date_range_rejects_reversed_boundaries():
2     with pytest.raises(ValueError):
3         DateRange(
4             date(2026, 5, 20),
5             date(2026, 5, 1),
6         )
```

With the Step 6 implementation, this test fails: construction succeeds. That is our next useful red test.

The rule can move into construction:

PYTHON

```
1 @dataclass
2 class DateRange:
3     opening_date: date
4     closing_date: date
5
6     def __post_init__(self):
7         if self.closing_date < self.opening_date:
8             raise ValueError(
9                 "closing date cannot be before opening date"
10            )
```

Now every DateRange created through its supported constructor satisfies the chronology rule.

That condition is an **invariant**.

MODELLING

Invariant

A condition that must remain true for every valid instance or accepted state of a model concept. For DateRange, the closing date must never precede the opening date.

Model decision: Chronology becomes a DateRange invariant

A DateRange is accepted only when its closing date is on or after its opening date. Callers no longer receive a DateRange and then decide whether its own boundaries are chronological.

The old `is_chronological()` method has now lost its purpose. If a DateRange exists, chronology is already guaranteed. Removing the method makes the model's promise clearer.

Model evolution

INITIAL INTERPRETATION

Create any pair of dates, then remember to ask whether the range is valid.

REVISED UNDERSTANDING

Allow `DateRange` construction only when its own invariant is satisfied.

— The invariant must survive after construction

Construction is only half the problem. With an ordinary dataclass, this is still possible:

PYTHON

```
1 period = DateRange(
2     date(2026, 5, 1),
3     date(2026, 7, 28),
4 )
5
6 period.closing_date = date(2026, 4, 1)
```

The constructor checked a valid value, but mutation later destroyed the invariant.

For this Value Object, the simplest design is to make the value **immutable**: once a `DateRange` has been created, its boundaries do not change.

Python's frozen dataclass gives us that behaviour:

PYTHON

```
1 @dataclass(frozen=True)
2 class DateRange:
3     opening_date: date
4     closing_date: date
5
6     def __post_init__(self):
7         if self.closing_date < self.opening_date:
8             raise ValueError(
9                 "closing date cannot be before opening date"
10            )
```

The choice fits the meaning discovered in Step 6. A `DateRange` is the value of its two boundaries. If a boundary changes, we have a *different DateRange value*, not the same value with a hidden history of mutations.

This does not mean that every Value Object in every language must be implemented with exactly this mechanism. It is a deliberate SchoolGrid design: immutability makes the `DateRange`'s value semantics and invariant easier to protect.

— Rescheduling replaces the value rather than editing it

Immutability does not prevent the school from changing a term's schedule. It changes how that modification is represented.

The original Term 2 period is:

PYTHON

```
1 original_period = DateRange(
2     date(2026, 5, 1),
3     date(2026, 7, 28),
```

MODELLING**Immutability**

An immutable object does not change its observable state after creation. A different value is represented by creating a different object.

```
4 )
```

If the school accepts the earlier opening date from Step 1, we construct a new value:

PYTHON

```
1 rescheduled_period = DateRange(
2     date(2026, 4, 15),
3     date(2026, 7, 28),
4 )
```

The original remains 1 May–28 July. The new value represents 15 April–28 July. A longer-lived concept such as Term 2 can later replace which `DateRange` it refers to without requiring the `DateRange` itself to mutate.

Example: A changed schedule uses a replacement value

Given the accepted Term 2 period is 1 May–28 July 2026.

When a reschedule to 15 April–28 July is accepted.

Then a new `DateRange` represents the new period while the original `DateRange` remains unchanged.

We have not yet named the DDD pattern that describes the continuing Term 2. That question is still waiting for a later step. For now, the important point is that **changing which value is used is different from mutating the value itself**.

— The administrator asks about a boundary day

Once a `DateRange` can be trusted to exist in a valid state, the next questions are not about validity. They are about useful behaviour.

The school has repeatedly treated opening and closing dates as inclusive. The administrator now asks a direct question about Term 2.

DOMAIN WORKSHOP

Does the closing day belong to the period?

School administrator / Developer

SA School administrator

Term 2 runs from 1 May through 28 July. If I ask whether 28 July belongs to Term 2, what does SchoolGrid answer?

DEV Developer

Yes. We agreed that both boundaries are inclusive.

SA School administrator

And 1 May?

DEV Developer

Also yes. A day before 1 May or after 28 July is outside the period.

The examples become tests:

PYTHON

```
1 def test_date_range_includes_both_boundaries():
2     period = DateRange(
3         date(2026, 5, 1),
4         date(2026, 7, 28),
5     )
6
7     assert period.includes(date(2026, 5, 1))
8     assert period.includes(date(2026, 7, 28))
```

The DateRange already has everything needed to answer the question:

PYTHON

```
1 def includes(self, day):
2     return self.opening_date <= day <= self.closing_date
```

The method reads almost like the business statement. The first and last days are inside because both comparisons are inclusive.

Example: A day just outside the period

Given a DateRange covers 1 May through 28 July 2026.

When we ask whether 29 July belongs to it.

Then includes returns false because 29 July lies beyond the inclusive closing boundary.

Inclusive boundaries also change how we count days

The administrator's next question looks trivial until we test the smallest boundary case.

How many days are in a DateRange whose opening and closing dates are both 20 May?

The answer is **one**, not zero. The single boundary day belongs to the range.

PYTHON

```
1 def test_same_day_range_has_duration_of_one_day():
2     period = DateRange(
3         date(2026, 5, 20),
4         date(2026, 5, 20),
5     )
6
7     assert period.days == 1
```

Python's subtraction counts the distance between the dates, so equal dates have a difference of zero. Our domain uses inclusive boundaries, therefore the domain calculation adds one:

PYTHON

```
1 @property
2 def days(self):
3     return (
4         self.closing_date - self.opening_date
5     ).days + 1
```

For 1 May through 2 May, the difference is one day but the inclusive period contains two calendar days.

The same formula gives two.

The important lesson is not the arithmetic. It is that a tiny implementation detail came from an explicit domain decision about what the boundaries mean.

— Then two periods meet at the same day

Step 2 established that two terms may not share a day. Before the `AcademicYear` can enforce that policy, we need a reliable way to answer a smaller factual question: **do these two `DateRanges` overlap?**

Consider these ranges:

PYTHON

```
1 term_1_period = DateRange(  
2     date(2026, 1, 1),  
3     date(2026, 3, 28),  
4 )  
5  
6 proposed_term_2_period = DateRange(  
7     date(2026, 3, 28),  
8     date(2026, 7, 28),  
9 )
```

They share 28 March. Because both `DateRanges` include their boundaries, they overlap.

A test captures that boundary case:

PYTHON

```
1 def test_ranges_overlap_when_they_share_a_boundary_day():  
2     first = DateRange(  
3         date(2026, 1, 1),  
4         date(2026, 3, 28),  
5     )  
6     second = DateRange(  
7         date(2026, 3, 28),  
8         date(2026, 7, 28),  
9     )  
10  
11     assert first.overlaps(second)  
12     assert second.overlaps(first)
```

The second assertion matters. Overlap is symmetric: if the first overlaps the second, the second overlaps the first.

The implementation can express the inclusive relationship directly:

PYTHON

```
1 def overlaps(self, other):  
2     return (  
3         self.opening_date <= other.closing_date  
4         and other.opening_date <= self.closing_date  
5     )
```

Now compare a period beginning the next day:

PYTHON

```

1 term_2_period = DateRange(
2     date(2026, 3, 29),
3     date(2026, 7, 28),
4 )
5
6 assert not term_1_period.overlaps(term_2_period)

```

This mirrors the boundary conversation from Step 1: 28 March conflicts; 29 March clears that particular relationship.

A DateRange can answer a fact without owning the policy

There is an important boundary in the model here.

`DateRange.overlaps(other)` answers a factual question about two date spans. It does *not* decide whether the school permits two Terms to use those spans.

The prohibition against overlapping Terms belongs to the academic calendar rule from BR-2.6. That decision needs term numbers, the terms being scheduled, and the wider `AcademicYear` context.

CONTRAST

DateRange responsibility

Answer: "Do these two inclusive spans share at least one day?"

AcademicYear responsibility

Decide: "May these two numbered terms coexist in this year's accepted schedule?"

This distinction prevents a useful Value Object from becoming a miniature calendar engine. It should provide the behaviour naturally associated with its value, while wider business policy remains with the concept that has the required context.

KEY IDEA A Value Object should protect its own invariants and provide behaviour that belongs naturally to its value. It should not absorb business decisions that require information it does not own.

The DateRange has now become a trustworthy building block

Putting the discoveries together gives us a compact implementation:

PYTHON

```

1 @dataclass(frozen=True)
2 class DateRange:
3     opening_date: date
4     closing_date: date
5
6     def __post_init__(self):
7         if self.closing_date < self.opening_date:
8             raise ValueError(
9                 "closing date cannot be before opening date"
10            )
11
12     @property
13     def days(self):

```

```

14         return (
15             self.closing_date - self.opening_date
16         ).days + 1
17
18     def includes(self, day):
19         return self.opening_date <= day <= self.closing_date
20
21     def overlaps(self, other):
22         return (
23             self.opening_date <= other.closing_date
24             and other.opening_date <= self.closing_date
25         )

```

Every part of this class now has a reason in the story:

Design choice	Domain reason
Value equality	Equal boundaries describe the same period.
Construction invariant	A period cannot close before it opens.
frozen=True	Changing a boundary means a different DateRange value.
days	Duration uses inclusive boundaries; a same-day range has one day.
includes	Opening and closing days both belong to the period.
overlaps	Sharing even one inclusive boundary day means the spans overlap.

The class is no longer just a convenient container for two dates. It carries a small, coherent piece of the domain model.

— One primitive is still leaking an invalid domain value

The next weak point is visible in the provisional Term from Step 5:

```

PYTHON
1  @dataclass
2  class Term:
3      number: int
4      period: DateRange | None = None

```

The type annotation says `int`, so Python is perfectly happy with this:

```

PYTHON
1  broken = Term(number=4)

```

But BR-2.2 says the SchoolGrid teaching model recognises only Term 1, Term 2, and Term 3.

We have just used a Value Object to stop invalid date spans from leaking through the model. The term number now presents the same kind of pressure in a smaller form: an unrestricted programming-language value represents states the domain does not permit.

Try it: Predict the DateRange behaviour

Without running the code, determine the result of each case and justify it using inclusive boundaries and the DateRange invariant:

1. 20 May–20 May: how many days?
2. Does 1 May–28 July include 1 May? Does it include 28 July?
3. Do 1 January–28 March and 28 March–20 May overlap?
4. Do 1 January–28 March and 29 March–20 May overlap?
5. What happens when construction is attempted with 20 May–1 May?
6. If the school reschedules a term, why should it create another DateRange rather than modify the accepted one?

For the overlap cases, separate the factual DateRange answer from the school's later decision about whether two Terms may use those ranges.

NEXT QUESTION

If SchoolGrid permits only term numbers 1, 2, and 3, why should an unrestricted integer such as 4 be allowed to enter the domain model at all?

Next: Step 8 – Another Value Object: Making Invalid Values Impossible

STEP 8

Another Value Object: Making Invalid Values Impossible

What you will discover

Recognise when an unrestricted programming-language value represents impossible domain states, model the permitted term numbers as a constrained Value Object, and distinguish a valid term number from the separate rule that prevents duplicate terms in one academic year.

Step 7 ended with a class that Python considers perfectly reasonable:

PYTHON

```
1 @dataclass
2 class Term:
3     number: int
4     period: DateRange | None = None
```

The annotation says that a term number is an integer. That is true, but it is not the whole business meaning.

Python is therefore equally happy with all of these:

PYTHON

```
1 Term(number=1)
2 Term(number=2)
3 Term(number=4)
4 Term(number=-10)
```

The first two make sense in SchoolGrid. The others do not. Our model still allows values that the school does not recognise.

An ordinary integer knows arithmetic, not the school rule

The developer returns to BR-2.2, which says that this teaching model permits only Term 1, Term 2, and Term 3.

DOMAIN WORKSHOP

Is 4 ever a term number here?

School administrator / Developer

DEV Developer

Python lets me construct a Term with number 4. Should that object ever enter the calendar model?

PYTHON

Type annotation

A type annotation documents the kind of value expected and helps static-analysis tools, but ordinary Python does not automatically enforce all business restrictions at runtime.

SA School administrator

No. We have Term 1, Term 2, and Term 3. There is no Term 4 in this academic calendar.

DEV Developer

What about 0 or a negative number?

SA School administrator

Those are not term numbers either.

DEV Developer

So the valid values are not “any integer.” They are exactly 1, 2, and 3?

SA School administrator

Exactly.

The programming language gives us a much larger set of values than the domain allows.

Model evolution

INITIAL INTERPRETATION

Represent a term number as any integer and validate it later.

REVISED UNDERSTANDING

Represent only the three values the SchoolGrid domain recognises.

A type annotation cannot enforce the business set

Changing the annotation does not solve the problem unless the new type actually protects its meaning.

This declaration:

PYTHON

```
1 @dataclass
2 class Term:
3     number: int
```

helps readers and static-analysis tools understand that `number` should be an integer. At runtime, however, `int` contains many values outside the business set `{1, 2, 3}`.

The business requirement is therefore stronger than “this value has integer syntax.” It is:

A term number is one of the three permitted positions in this academic year: 1, 2, or 3.

That is a small domain concept with its own invariant.

Make the invalid example executable first

We already know the boundary we want to protect. A useful next test says that 4 must not become a `TermNumber`:

MODELLING

Constrained value

A value whose business meaning permits only a subset of the values its programming-language type could otherwise represent.

PYTHON

```

1 def test_term_number_rejects_four():
2     with pytest.raises(ValueError):
3         TermNumber(4)

```

We should also protect the lower side of the set:

PYTHON

```

1 def test_term_number_rejects_zero():
2     with pytest.raises(ValueError):
3         TermNumber(0)

```

The first implementation can begin as the smallest real type that lets the test reach the behaviour we care about:

PYTHON

```

1 @dataclass(frozen=True, order=True)
2 class TermNumber:
3     value: int

```

Now `TermNumber(4)` can be constructed, so the rejection test fails for the right reason: the business restriction is missing.

We add that restriction at construction:

PYTHON

```

1 def __post_init__(self):
2     if self.value not in (1, 2, 3):
3         raise ValueError(
4             "term number must be 1, 2, or 3"
5         )

```

Construction is now the boundary at which the domain restriction is enforced. If a `TermNumber` exists, its value is one the school permits.

Model decision: `TermNumber` represents the constrained domain value

SchoolGrid does not pass arbitrary integers around as term numbers. A supported `TermNumber` can contain only 1, 2, or 3, so code receiving one does not need to repeat that validation.

The three accepted values deserve tests too

Rejecting bad input is only half the behaviour. A broken implementation that rejects every integer would satisfy both tests above.

We therefore record the three accepted positions:

PYTHON

```

1 @pytest.mark.parametrize("value", [1, 2, 3])
2 def test_term_number_accepts_permitted_values(value):
3     number = TermNumber(value)
4
5     assert number.value == value

```

Now the executable examples describe both sides of the rule:

Input	Expected domain result
1	Construct TermNumber 1
2	Construct TermNumber 2
3	Construct TermNumber 3
0	Reject
4	Reject
-1	Reject

The class is small because the business concept is small. That is not a weakness. The purpose is not to justify a class with lots of methods. The purpose is to make an invalid domain value difficult to represent accidentally.

KEY IDEA A useful Value Object can be tiny. Its value comes from protecting domain meaning, not from having many fields or methods.

Term can now require the meaningful value

The provisional Term now communicates that its supported number is a TermNumber, not an arbitrary integer:

PYTHON

```
1 @dataclass
2 class Term:
3     number: TermNumber
4     period: DateRange | None = None
```

Construction now reads differently:

PYTHON

```
1 term_2 = Term(number=TermNumber(2))
```

There is a little more syntax than `Term(number=2)`, but the model carries more information. Any code that already has a TermNumber knows that the value is one of the permitted positions.

This does not mean a Python annotation magically prevents a caller from writing `Term(number=4)`. The domain's supported construction path should supply a TermNumber; later layers can enforce that boundary when raw input enters the system.

Compare that with the old approach:

PYTHON

```
1 def schedule_term(number: int):
2     if number not in (1, 2, 3):
3         raise ValueError("invalid term number")
4     ...
```

If every operation accepts an unrestricted integer, every operation becomes a possible place to forget the rule. The constrained value moves the invariant to one supported construction boundary.

PYTHON

Annotation vs runtime rule

The annotation communicates intent. The TermNumber constructor enforces the business restriction. They solve different problems.

— This is another Value Object, but for a different reason

`DateRange` and `TermNumber` look different in size, but they share the important Value Object characteristics we discovered earlier.

Two independently constructed `TermNumber(2)` values mean the same thing:

PYTHON

```
1 first = TermNumber(2)
2 second = TermNumber(2)
3
4 assert first == second
```

We do not need an identifier to distinguish “this 2” from “that 2.” Their meaning comes from the value they contain.

The object is also immutable. Turning `TermNumber 2` into `TermNumber 3` would not be “editing the same value”; it would mean using a different numbered position.

DDD discovery: Another Value Object

`TermNumber` is a Value Object because its meaning is determined by its permitted value. Equality is by value, it has no separate business identity, and its construction protects the invariant that only 1, 2, and 3 are valid.

This reinforces an important point: Value Objects do not have to be composite structures such as `DateRange`. A single constrained value can deserve a domain type when the restriction carries important meaning.

— But do not wrap every primitive automatically

It would be easy to turn this lesson into a mechanical rule: every integer, string, and date should receive its own class.

That would miss the reason `TermNumber` appeared.

We introduced it because all of these were true:

- the business has a precise restricted set of valid values;
- an ordinary `int` allows many invalid states;
- the restriction appears wherever a term number enters the domain;
- the value has a clear name in the ubiquitous language;
- construction can protect the rule once for all callers.

A primitive that carries no additional domain meaning may remain a primitive. The model should become more expressive where the business demands it, not more ceremonial everywhere.

CONTRAST**Mechanical wrapping**

Create a class merely because a field uses an int, string, or date.

Domain-driven value

Create a meaningful type when the domain gives the value constraints or behaviour worth protecting.

Ordering comes from the meaning of the number

The three term numbers also describe positions in a sequence. In the implementation above, `order=True` lets that relationship be expressed naturally:

PYTHON

```
1 assert TermNumber(1) < TermNumber(2)
2 assert TermNumber(2) < TermNumber(3)
```

That ordering will later help us state rules such as “an earlier-numbered term must finish before a later-numbered term begins.”

The comparison does not decide whether two Terms overlap or whether a schedule is acceptable. It only captures a fact about the ordered values 1, 2, and 3. Wider calendar policy still belongs with the concept that can see the calendar.

A valid TermNumber does not mean the term may be added

Now we reach an important boundary. This is valid:

PYTHON

```
1 number = TermNumber(2)
```

And this is also valid:

PYTHON

```
1 another_number = TermNumber(2)
```

The two values are equal. Nothing is wrong with constructing `TermNumber 2` twice.

But BR-2.3 says one `AcademicYear` may not contain two scheduled Terms with number 2.

Those are different rules.

DOMAIN WORKSHOP**If 2 is valid twice, why is a second Term 2 wrong?**

School administrator / Developer

**Developer**

I can construct `TermNumber(2)` twice and both values are valid. Does that mean the year may contain two Term 2s?

SA School administrator

No. The number 2 is a valid value every time you write it. The problem is having two different Terms claiming the Term 2 position in the same academic year.

DEV Developer

So one rule validates the value itself, while another rule validates how Terms are combined?

SA School administrator

Exactly. Do not make the number responsible for knowing the whole calendar.

Example: Valid value, invalid calendar combination

Given Academic Year 2026 already contains a Term whose number is TermNumber 2.

When someone attempts to add another Term whose number is also TermNumber 2.

Then TermNumber 2 itself is still a valid value, but the AcademicYear must reject the duplicate Term.

If TermNumber tried to prevent a second TermNumber(2) from ever existing, it would need global knowledge of every object already created. That is not the meaning of the value and would make the model much worse.

CONTRAST**TermNumber invariant**

The value must be 1, 2, or 3.

AcademicYear invariant

Within one academic year, the same term number must not identify two Terms.

The first rule is local to one value. The second requires the collection of Terms inside an AcademicYear.

MODELLING**Local vs contextual invariant**

A local invariant can be decided from one object or value. A contextual invariant needs information from the surrounding model.

The term number identifies something without being that thing

A subtle question now appears. If the school says “Term 2,” the number 2 helps us identify which term is being discussed. Does that make TermNumber itself an entity?

No. The value 2 has no independent history. Two TermNumber(2) objects are interchangeable. We do not care which instance was created first.

The *Term*, however, is beginning to behave differently.

Recall the reschedule from earlier steps:

PYTHON

```
1 term_2 = Term(
2     number=TermNumber(2),
3     period=DateRange(
4         date(2026, 5, 1),
5         date(2026, 7, 28),
6     ),
7 )
```

Later, the school accepts a new period beginning on 15 April. The DateRange value changes. The TermNumber remains 2. More importantly, the administrator still speaks about the continuing **Term 2**

rather than a newly created term.

That continuity is not a property of the integer-like value 2. It belongs to the Term that 2 identifies within its academic year.

Example: The identifying value stays while other state changes

Given Term 2 of Academic Year 2026 has an accepted period of 1 May–28 July.

When the school reschedules it to 15 April–28 July.

Then its DateRange is a different value, its TermNumber remains 2, and the school still treats it as the same Term 2.

We have now reached a question that Value Object semantics cannot answer on their own: how should the model represent a domain concept that remains the same thing while some of its attributes change?

Try it: Separate value validity from calendar validity

For each statement, decide whether it is a rule about one TermNumber or a rule requiring wider AcademicYear context. Explain why.

1. TermNumber 4 is proposed.
2. TermNumber 2 is constructed twice in two different tests.
3. Academic Year 2026 already contains Term 2 and another Term 2 is proposed.
4. TermNumber 1 is compared with TermNumber 3 to determine their order.
5. Term 2 changes from one valid DateRange to another valid DateRange.

Then answer: if the value 2 is interchangeable wherever it appears, what exactly is it that continues through a reschedule?

NEXT QUESTION

Term 2 can receive a different DateRange and still remain the same Term 2. What kind of domain object is defined by that continuing identity rather than by equality of all its current attributes?

Next: Step 9 – Entities: When Identity Matters

STEP 9

Entities: When Identity Matters

What you will discover

Recognise a domain concept whose identity continues while its current attributes change, distinguish entity identity from Value Object equality, and explain why Term identity is scoped to its AcademicYear rather than made globally unique by an invented identifier.

Step 8 ended with a question that our Value Objects cannot answer. Term 2 begins with one period:

PYTHON

```
1 term_2 = Term(
2     number=TermNumber(2),
3     period=DateRange(
4         date(2026, 5, 1),
5         date(2026, 7, 28),
6     ),
7 )
```

The school then accepts an earlier opening date. The period becomes 15 April–28 July.

The old and new DateRanges are different values. Yet the administrator never speaks about deleting one term and creating another. She says: “**reschedule Term 2.**”

Something is continuing while one of its attributes changes.

Ask what survived the change

The developer makes that continuity explicit.

DOMAIN WORKSHOP

Is this still the same Term 2?

School administrator / Developer

DEV Developer

Before the change, Term 2 runs from 1 May to 28 July. After the change, it runs from 15 April to 28 July. Are those two different terms?

SA School administrator

No. It is the same Term 2 with revised dates.

DEV Developer

So the period can change without changing which term we are talking about?

SA School administrator

Exactly.

DEV Developer

What would make it a different term?

SA School administrator

If we were talking about Term 1 or Term 3, that would be a different term. But changing Term 2's dates does not turn it into another term.

The `DateRange` lesson from Step 6 now gives us a useful contrast.

For a `DateRange`, changing one boundary means a different value. For a `Term`, changing its period does *not* mean a different `Term`.

Model evolution

INITIAL INTERPRETATION

Assume every domain object is defined by all of its current values.

REVISED UNDERSTANDING

Ask whether the business tracks a continuing thing whose attributes may change over time.

Value equality is the wrong question for Term

A `DateRange` is equal to another `DateRange` when their boundaries are equal:

PYTHON

```
1 first = DateRange(
2     date(2026, 5, 1),
3     date(2026, 7, 28),
4 )
5 second = DateRange(
6     date(2026, 5, 1),
7     date(2026, 7, 28),
8 )
9
10 assert first == second
```

That works because the boundaries completely describe the domain value.

If we apply the same rule mechanically to `Term`, we get into trouble. Imagine we compare every current attribute:

PYTHON

```
1 before = Term(
2     TermNumber(2),
3     DateRange(
4         date(2026, 5, 1),
5         date(2026, 7, 28),
6     ),
7 )
8
```

```

9  after = Term(
10      TermNumber(2),
11      DateRange(
12          date(2026, 4, 15),
13          date(2026, 7, 28),
14      ),
15  )

```

An all-fields comparison would say these are different because the periods differ. The school says they can represent the same continuing Term 2 at different points in its life.

The question is therefore no longer:

Do all of the current attributes have equal values?

It is:

Are we talking about the same domain thing, even though some of its current state has changed?

Now the Entity concept has earned a name

Domain-Driven Design calls a domain object whose continuity is defined by **identity** rather than by equality of all current attributes an **Entity**.

DDD CONCEPT

Entity

A domain object distinguished by a continuing identity. Its state may change over time while the business still regards it as the same thing.

DDD discovery: Entity

Term is an Entity in the SchoolGrid model because the school tracks Term 1, Term 2, and Term 3 as continuing concepts. A Term's period may change, but a valid reschedule does not create a new Term identity.

This does not mean that Entities must always be mutable in every possible way. It means that changes to some attributes do not determine whether the Entity is the same thing.

CONTRAST

DateRange – Value Object

Its current boundary values define what it is. Different boundaries mean a different value.

Term – Entity

Its identity continues while its current period may change.

The distinction comes from the domain, not from whether Python happens to use a class for both.

What identifies the Term?

The next question is unavoidable. If Term is an Entity, what identity does the school actually use?

The conversation so far already contains the answer: *Term 1*, *Term 2*, and *Term 3*. Within one AcademicYear, the TermNumber distinguishes the Terms.

DOMAIN WORKSHOP

Is TermNumber enough everywhere?

School administrator / Developer

DEV Developer

Within Academic Year 2026, does the number 2 tell us which term you mean?

SA School administrator

Yes. There is only one Term 2 in that academic year.

DEV Developer

But Academic Year 2027 will also have a Term 2. Is that the same term as Term 2 of 2026?

SA School administrator

No. Those belong to different academic years.

DEV Developer

So "2" identifies the Term only inside the academic year that owns it?

SA School administrator

Correct.

This gives us a **scoped identity**.

Conceptually, these identities are different:

PYTHON

```

1 term_2_of_2026 = (2026, TermNumber(2))
2 term_2_of_2027 = (2027, TermNumber(2))
3
4 assert term_2_of_2026 != term_2_of_2027

```

The tuple is not our final model. It simply makes the scope visible: the same TermNumber can participate in different Term identities because the owning AcademicYear is different.

Example: The same number in different years**Given** Academic Year 2026 has Term 2 and Academic Year 2027 also has Term 2.**When** the two Terms are identified.**Then** they are different Entities because each TermNumber is interpreted within a different AcademicYear.

This is also why BR-2.3 says that a term number may occur at most once *within one academic year*. Reusing TermNumber 2 in another year is expected.

MODELLING

Identity scope

The context within which an identifying value is unique. TermNumber 2 identifies one Term inside a particular AcademicYear, but the value 2 is reused in other academic years.

— We do not need to invent a global ID

Once the word *Entity* appears, software teams often reach immediately for a UUID or database primary key:

```
PYTHON
1 @dataclass
2 class Term:
3     id: str
4     number: TermNumber
5     period: DateRange | None = None
```

There is nothing technically impossible about such a design. The problem is that we have not discovered a business need for that identifier.

The school already has an identity rule that makes sense in its language:

Within an AcademicYear, the TermNumber identifies the Term.

READING CUE

Persistence and identity

A database may later use a surrogate key for persistence. That storage choice should not automatically replace the identity that the domain uses.

Model decision: Term identity is contextual

A Term is identified by its TermNumber within its owning AcademicYear. SchoolGrid does not introduce a global domain ID merely because Term is an Entity.

This decision may change if a later requirement creates a real need for a global identifier. Domain models are allowed to evolve with evidence.

— TermNumber identifies the Entity without becoming an Entity

Step 8 deliberately separated TermNumber from Term. That separation matters even more now.

These two values are interchangeable:

```
PYTHON
1 first = TermNumber(2)
2 second = TermNumber(2)
3
4 assert first == second
```

There is no meaningful history of “the first 2” versus “the second 2.” They are equal Value Objects.

But Term 2 has continuity. It can be unscheduled, scheduled, and later rescheduled while remaining the same Term within its year.

The identifying value and the Entity it identifies therefore have different semantics:

Concept	What makes it equal/same?	Can current state vary?
DateRange	Equal boundary values	No; changed boundaries mean another value
TermNumber	Equal numeric value	No; 2 changed to 3 is another value
Term	Same identity within its AcademicYear	Yes; its current period may change

READING CUE

Identifier versus Entity

An identifier can itself be a Value Object. That does not turn the identifier into the Entity.

Do not confuse Entity identity with Python object identity

There is another trap. Python's `is` operator asks whether two variables point to the same object in memory:

PYTHON

```
1 term is same_reference
```

That is a programming-language question, not automatically a domain identity question.

Imagine SchoolGrid saves Term 2, the process ends, and tomorrow the term is loaded again from storage. The new Python object did not exist yesterday. It can still represent the same domain Entity: Term 2 of Academic Year 2026.

Misconception: Python objects and domain identity

Likewise, two separate Python objects could accidentally claim the same scoped identity. The domain would not thereby gain two legitimate Term 2 Entities; it would reveal that our model or persistence process had allowed duplicate representations of the same identity.

KEY IDEA Entity identity belongs to the domain. Memory addresses, object references, and database keys may help implement that identity, but they do not define it unless the domain gives them that meaning.

Make the current Python model stop pretending Term is a value

Python's `@dataclass` generates equality from all fields by default. That was useful for `DateRange` and `TermNumber` because their fields define their values. It is misleading for `Term` because its period may change without changing its identity.

For now we remove generated value equality:

PYTHON

```
1 @dataclass(eq=False)
2 class Term:
3     number: TermNumber
4     period: DateRange | None = None
```

This does *not* finish Entity equality for us. It simply prevents Python from silently teaching the wrong rule: "two Terms are equal exactly when all fields match."

Because Term identity is scoped by AcademicYear, the owner will eventually be responsible for distinguishing and locating its Terms by TermNumber. We do not need to force a context-free `Term.__eq__` implementation before that relationship is properly modelled.

A simple continuity test

We can still make the business continuity observable without pretending that all Entity identity lives inside Python's equality operator.

Start with Term 2 and remember its identifying value:

PYTHON

```
1 def test_term_keeps_its_number_when_period_changes():
2     term = Term(
3         TermNumber(2),
4         DateRange(
5             date(2026, 5, 1),
6             date(2026, 7, 28),
7         ),
8     )
9     identity_within_year = term.number
10
11     term.period = DateRange(
12         date(2026, 4, 15),
13         date(2026, 7, 28),
14     )
15
16     assert term.number == identity_within_year
```

The test demonstrates the point we need in this step: the current period changed, but the TermNumber that identifies the Entity within its AcademicYear did not.

The direct assignment to `term.period` is intentionally uncomfortable. It exposes the next design problem rather than hiding it.

Identity alone does not protect state changes

We now know what Term is, but we still have weak behaviour:

PYTHON

```
1 term.period = another_period
```

Any caller can replace the period directly. Nothing in that assignment says whether the action is an initial schedule or a reschedule. Nothing asks whether other calendar rules must be checked. Nothing communicates the business intent.

Worse, generic field mutation makes it easy to bypass the language we worked so hard to establish:

PYTHON

```
1 term.period = new_period
```

versus the business statement:

Reschedule Term 2 to a proposed period.

Knowing that Term is an Entity therefore creates a second question: how should an Entity expose changes to its state without becoming a bag of writable fields?

Try it: Find the identity, not the matching attributes

For each case, decide whether the evidence suggests the same Entity, equal Value Objects, or different domain things. Explain the identity scope you are using.

1. Two independently constructed DateRanges both cover 1 May–28 July 2026.
2. Term 2 of Academic Year 2026 changes from 1 May–28 July to 15 April–28 July.
3. Term 2 of Academic Year 2026 and Term 2 of Academic Year 2027.
4. Term 1 and Term 2 happen to be given identical DateRanges in a broken calendar.
5. A Term 2 object is saved today and reconstructed from persistence tomorrow with the same AcademicYear and TermNumber.

Then explain why neither `is` nor equality of all dataclass fields is a safe universal definition of Entity identity.

NEXT QUESTION

If Term is a continuing Entity, should any caller be allowed to replace its fields directly, or should changes such as scheduling and rescheduling be expressed as meaningful domain behaviour?

Next: Step 10 – Entity State and Behaviour

STEP 10

Entity State and Behaviour

What you will discover

Replace unrestricted Entity mutation with intention-revealing behaviour, distinguish observations from state-changing operations, and identify which rules a Term can enforce using its own state and which rules require wider AcademicYear context.

Step 9 established that Term is an Entity. Its TermNumber identifies it within an AcademicYear, while its current DateRange may change without creating a new Term.

But the implementation ended in an uncomfortable place:

PYTHON

```
1 term.period = new_period
```

That line changes state, but it says almost nothing about the business action that caused the change. Was the Term previously unscheduled? Is this its first schedule? Is this a reschedule? Is the caller even allowed to replace the period?

The model now knows *what* Term is, but it does not yet control *how* Term changes.

A writable field hides the business event

The developer shows the assignment to the administrator.

DOMAIN WORKSHOP

What does “set the period” mean?

School administrator / Developer

DEV Developer

When the dates change, I can simply assign a new DateRange to the Term's period.

SA School administrator

But why are the dates changing?

DEV Developer

The code does not say. It just replaces the field.

SA School administrator

That matters. If the Term has never had dates, we are scheduling it. If it already has dates, we are rescheduling it.

DEV Developer

So the same field assignment is hiding two different business actions?

SA School administrator

Exactly. And I would not expect someone to “reschedule” a Term that has never been scheduled.

The distinction was already present in the ubiquitous language from Step 4. The code has simply failed to preserve it.

Model evolution**INITIAL INTERPRETATION**

Expose writable state and let callers decide what an assignment means.

REVISED UNDERSTANDING

Expose operations that state the business intention and let the Entity protect rules it can decide locally.

— Begin with the Entity’s possible local states

Because SchoolGrid allows an AcademicYear to be configured progressively, a Term may exist before it has dates. We therefore have at least two meaningful local states:

Term state	Meaning
Unscheduled	The Term exists but does not yet have an accepted DateRange.
Scheduled	The Term currently has an accepted DateRange.

We do not need a separate status field to represent this yet. The existing period already tells us which state the Term is in.

Instead of accepting the period directly through a public constructor field, we can let a new Term begin unscheduled:

PYTHON

```
1 class Term:
2     def __init__(self, number):
3         self._number = number
4         self._period = None
```

The TermNumber should also not be casually replaced. It participates in the Term’s identity within its AcademicYear. We therefore expose observations through read-only properties:

PYTHON

```
1 @property
2 def number(self):
3     return self._number
4
5 @property
6 def period(self):
```

READING CUE**Python conventions**

The leading underscore is a Python convention indicating that these attributes are internal implementation details. It is not an impenetrable security boundary. The domain API we design around them is what communicates the supported way to work with the Entity.

```
7         return self._period
```

Now callers can ask which Term this is and inspect its current period without a public setter advertising unrestricted mutation.

— Observation does not have to expose mutation

The first useful behaviour does not change anything. We often need to ask whether a Term has been scheduled yet.

PYTHON

```
1 @property
2 def is_scheduled(self):
3     return self._period is not None
```

A test records the initial state:

PYTHON

```
1 def test_new_term_starts_unscheduled():
2     term = Term(TermNumber(2))
3
4     assert not term.is_scheduled
5     assert term.period is None
```

This is an **observation**: it reports something about the Entity without changing its state.

That distinction will help us keep tests focused. A test that wants to know whether a Term is scheduled should ask the model directly rather than infer the answer from incidental storage details elsewhere.

— The first state change is scheduling

The administrator has already defined *schedule*: give a Term its period when it is not yet scheduled.

We can make that operation explicit:

PYTHON

```
1 def schedule(self, period):
2     if self.is_scheduled:
3         raise ValueError("term is already scheduled")
4
5     self._period = period
```

The simplest successful example now reads like the school action:

PYTHON

```
1 def test_schedule_assigns_the_first_period():
2     term = Term(TermNumber(2))
3     period = DateRange(
4         date(2026, 5, 1),
5         date(2026, 7, 28),
6     )
7
8     term.schedule(period)
```

```

9
10     assert term.is_scheduled
11     assert term.period == period

```

Compare that with a generic assignment:

PYTHON

```

1 term.period = period

```

The method call carries substantially more domain meaning:

PYTHON

```

1 term.schedule(period)

```

The difference is not cosmetic. `schedule` can also protect the local transition rule that an already scheduled Term cannot be scheduled for the first time again.

— A rejected local transition must preserve the current state

Suppose Term 2 already has its accepted period. Calling `schedule` again is not another initial schedule.

A test should verify both rejection and preservation of the existing state:

PYTHON

```

1 def test_schedule_rejects_an_already_scheduled_term():
2     term = Term(TermNumber(2))
3     original = DateRange(
4         date(2026, 5, 1),
5         date(2026, 7, 28),
6     )
7     term.schedule(original)
8
9     with pytest.raises(ValueError):
10         term.schedule(
11             DateRange(
12                 date(2026, 4, 15),
13                 date(2026, 7, 28),
14             )
15         )
16
17     assert term.period == original

```

Here the Term has enough information to enforce the rule by itself: it knows whether it already has a period.

Model decision: Term owns its local scheduling transition

An unscheduled Term may receive its first `DateRange` through `schedule`. A Term that already has a period rejects another initial scheduling attempt. This decision requires only the Term's own current state.

READING CUE

Rejection preserves state

The final assertion matters. A rejected operation should not leave the Entity half changed.

— Changing an existing schedule deserves a different verb

When Term 2 already has dates, the school does not call the next change *schedule*. It calls it **reschedule**.

The local state transition can therefore be expressed separately:

PYTHON

```
1 def reschedule(self, new_period):
2     if not self.is_scheduled:
3         raise ValueError("term is not scheduled")
4
5     self._period = new_period
```

This method can reject a misuse that requires no information outside the Term:

PYTHON

```
1 def test_unscheduled_term_cannot_be_rescheduled():
2     term = Term(TermNumber(2))
3
4     with pytest.raises(ValueError):
5         term.reschedule(
6             DateRange(
7                 date(2026, 4, 15),
8                 date(2026, 7, 28),
9             )
10        )
11
12     assert term.period is None
```

Once the Term is scheduled, a local reschedule replaces its current DateRange while its identifying TermNumber remains unchanged:

PYTHON

```
1 def test_reschedule_changes_period_not_identity():
2     term = Term(TermNumber(2))
3     term.schedule(
4         DateRange(
5             date(2026, 5, 1),
6             date(2026, 7, 28),
7         )
8     )
9
10    term.reschedule(
11        DateRange(
12            date(2026, 4, 15),
13            date(2026, 7, 28),
14        )
15    )
16
17    assert term.number == TermNumber(2)
18    assert term.period == DateRange(
19        date(2026, 4, 15),
20        date(2026, 7, 28),
21    )
```

This is the Entity behaviour Step 9 was preparing us for: the Entity continues while part of its current

state changes.

— An Entity can also answer questions about itself

Behaviour is not limited to commands that modify state. A Term can answer useful questions using the Value Objects it owns.

Suppose the attendance subsystem later asks whether Term 2 occurs on 28 July. The Term does not need to expose date arithmetic to the caller. It can delegate the factual date question to its DateRange:

PYTHON

```
1 def occurs_on(self, day):
2     if not self.is_scheduled:
3         return False
4
5     return self._period.includes(day)
```

The test remains expressed in domain language:

PYTHON

```
1 def test_scheduled_term_occurs_on_its_closing_day():
2     term = Term(TermNumber(2))
3     term.schedule(
4         DateRange(
5             date(2026, 5, 1),
6             date(2026, 7, 28),
7         )
8     )
9
10    assert term.occurs_on(date(2026, 7, 28))
```

There is a useful division of responsibility here. Term answers the domain question “does this Term occur on this day?” while DateRange performs the inclusive membership calculation.

KEY IDEA Rich domain behaviour does not mean placing every rule in one class. A Term should express questions and transitions that belong to the Term, while delegating smaller value calculations to the Value Objects that already know how to answer them.

— Intention-revealing methods are not just renamed setters

It would be easy to write this and claim victory:

PYTHON

```
1 def set_period(self, period):
2     self._period = period
```

Misconception: A method is not automatically domain behaviour

That method hides the assignment behind a function call, but it still does not say why the state is changing or which transition rules apply.

Compare the vocabulary:

Operation	What the name communicates
<code>set_period(period)</code>	A technical field is being replaced. Business intention remains unclear.
<code>schedule(period)</code>	An unscheduled Term is receiving its first accepted period.
<code>reschedule(period)</code>	A scheduled Term is moving to a different accepted period.
<code>occurs_on(day)</code>	A question is being asked about whether this Term includes a day.

An **intention-revealing operation** makes the reason for the action visible in the model.

The benefit is deeper than readability. Different intentions can carry different preconditions, rules, and consequences.

The Term can protect only what it knows

At this point the design looks stronger. A caller cannot use the supported API to reschedule an unscheduled Term, and a second initial schedule is rejected.

Now consider the boundary example that has followed us since Step 1.

Term 1 is already scheduled through 28 March:

PYTHON

```
1 term_1 = Term(TermNumber(1))
2 term_1.schedule(
3     DateRange(
4         date(2026, 1, 1),
5         date(2026, 3, 28),
6     )
7 )
```

Term 2 currently runs from 1 May through 28 July:

PYTHON

```
1 term_2 = Term(TermNumber(2))
2 term_2.schedule(
3     DateRange(
4         date(2026, 5, 1),
5         date(2026, 7, 28),
6     )
7 )
```

Now somebody proposes this reschedule:

PYTHON

```
1 term_2.reschedule(
2     DateRange(
3         date(2026, 3, 28),
4         date(2026, 7, 28),
5     )
6 )
```

From Term 2's local point of view, nothing is obviously wrong:

- Term 2 is already scheduled, so rescheduling is a meaningful transition;
- the proposed DateRange is chronological;

MODELLING

Intention-revealing operation

A method whose name and contract express the domain action or question being performed rather than merely exposing the object's storage mechanics.

- TermNumber 2 remains unchanged.

Yet the calendar is wrong. Term 1 and Term 2 now both include 28 March.

— Giving Term more methods will not give it missing information

We could try to make `Term.reschedule` reject the conflict, but what would it compare against?

Term 2 currently knows its own number and its own period. It does not know the period of Term 1 or Term 3. It does not know which other Terms belong to Academic Year 2026. It therefore lacks the information required to enforce BR-2.6.

Passing all sibling terms into every Entity method would technically make the information available:

PYTHON

```
1 term_2.reschedule(new_period, all_other_terms)
```

But that signature is telling us something important: the decision is no longer about Term 2 alone. We would be forcing a wider calendar responsibility into one child Entity.

CONTRAST

Local Term rule

“Can an unscheduled Term be rescheduled?”
Term can answer from its own state.

Calendar rule

“Would this proposed Term 2 period conflict with another Term?” The answer requires the other Terms in the AcademicYear.

The information needed by a rule is again showing us where responsibility cannot live.

— A locally valid operation can still be globally unacceptable

This is the same distinction we first saw with `DateRange`, now at the Entity level.

A `DateRange` may be internally valid. A Term may be in a state where rescheduling is locally meaningful. The proposed change can still be unacceptable when the Term is considered together with the rest of the AcademicYear.

Example: The Entity cannot see the conflict

Given Term 1 ends on 28 March and Term 2 is already scheduled later in the year.

When Term 2 is proposed to be rescheduled so that it opens on 28 March.

Then the proposed `DateRange` is valid and Term 2 is locally allowed to be rescheduled, but the complete calendar must reject the change because both Terms would claim the same day.

This exposes a limitation in our current public `reschedule` operation. The method is capable of applying a locally meaningful state change before the wider calendar has approved it.

We should not solve that problem by teaching Term about every other Term. Instead, we need to discover the model boundary that owns the information required to approve the change before the Entity is mutated.

Tests should use behaviour, not reach through the model

The shift from writable fields to supported operations also changes how we test. A test should prefer this:

PYTHON

```
1 term.schedule(period)
2
3 assert term.is_scheduled
4 assert term.period == period
```

over reaching into the Entity's internal representation:

PYTHON

```
1 term._period = period
2 assert term._period == period
```

The second test is tightly coupled to the current storage choice. It can remain green even while bypassing every domain transition we intended the Entity to protect.

Tests should observe supported outcomes: current period, scheduling status, identity, occurrence on a date, or a raised domain error. That gives the implementation room to change without weakening the business contract.

Model decision: Expose Entity behaviour, not general-purpose mutation

Term exposes domain questions and state transitions such as `is_scheduled`, `occurs_on`, `schedule`, and `reschedule`. Its internal attributes are not the public business API. Rules requiring other Terms remain outside Term because the Entity does not have the information needed to decide them.

What we have gained—and what is still missing

Term now has a clearer role in the model:

Term can decide	Term cannot decide alone
Whether it currently has a period	Whether another Term has the same number
Whether an initial schedule is being attempted twice	Whether its proposed period overlaps another Term
Whether an unscheduled Term is being rescheduled	Whether its dates comply with the AcademicYear's wider calendar policy
Whether it occurs on a given day	Whether the complete year remains chronologically consistent

The right-hand column is now the important one. Several business rules are not owned by any individual `DateRange` or `Term` because their truth emerges only when otherwise valid objects are considered together.

That is the pressure that moves us into the next part of the book.

Try it: Separate local behaviour from coordinating behaviour

For each proposed operation, decide whether Term has enough information to handle it by itself. If not, state what additional information is missing.

1. Ask whether an unscheduled Term has a current period.
2. Give an unscheduled Term its first valid DateRange.
3. Reschedule a Term that has never been scheduled.
4. Ask whether Term 2 occurs on 28 July.
5. Reschedule Term 2 to begin on 28 March when Term 1 ends on 28 March.
6. Add a second Term 2 to Academic Year 2026.
7. Reschedule Term 3 to close in January 2027 under SchoolGrid's current 2026 calendar-year policy.

Then explain why adding more code to Term cannot solve a rule when the Entity does not possess the information required to evaluate that rule.

NEXT QUESTION

If a DateRange can be valid and a Term can be locally valid, yet their combination with other Terms can still make the calendar invalid, what part of the model should protect consistency across those related objects?

Next: Step 11 – When Valid Objects Become Invalid Together

STEP 11

When Valid Objects Become Invalid Together

What you will discover

Distinguish local object validity from consistency among related objects, identify the information required to validate a calendar-wide change, and explain why neither `DateRange` nor an individual `Term` can protect rules that depend on sibling `Terms`.

Step 10 ended with a troubling result. Our smaller objects can all be behaving correctly and the school calendar can still become wrong.

Term 1 is scheduled from 1 January through 28 March:

PYTHON

```
1 term_1 = Term(TermNumber(1))
2 term_1.schedule(
3     DateRange(
4         date(2026, 1, 1),
5         date(2026, 3, 28),
6     )
7 )
```

Term 2 is scheduled from 1 May through 28 July:

PYTHON

```
1 term_2 = Term(TermNumber(2))
2 term_2.schedule(
3     DateRange(
4         date(2026, 5, 1),
5         date(2026, 7, 28),
6     )
7 )
```

Now someone proposes moving Term 2 so that it begins on 28 March:

PYTHON

```
1 proposed = DateRange(
2     date(2026, 3, 28),
3     date(2026, 7, 28),
4 )
```

Nothing inside that `DateRange` is invalid. It opens before it closes. Term 2 is already scheduled, so a reschedule is locally meaningful. `TermNumber` 2 is still a valid term number.

Yet the proposed calendar is unacceptable.

— The school sees a conflict that no individual object sees

The developer walks through the proposal with the administrator.

DOMAIN WORKSHOP

Everything is valid. Why is the calendar wrong?

School administrator / Developer

DEV Developer

The new Term 2 period is a valid DateRange. Term 2 is allowed to be rescheduled. Its number is still valid. Why should the request fail?

SA School administrator

Because Term 1 already includes 28 March. Term 2 cannot also begin on that day.

DEV Developer

So neither object is broken by itself?

SA School administrator

Correct. The problem appears when you put the two Terms together in the same academic year.

DEV Developer

And if Term 2 began on 29 March instead?

SA School administrator

Then it would no longer share that day with Term 1. We would still check the other rules, but that particular conflict would be gone.

This is a different kind of modelling problem from the ones we solved in Steps 6 through 10.

A DateRange could protect its own chronology. A TermNumber could protect its own allowed values. A Term could protect whether scheduling or rescheduling made sense for its own state.

Now the truth of the rule depends on **several individually valid objects considered together**.

Model evolution

INITIAL INTERPRETATION

Ask whether each object is valid in isolation.

REVISED UNDERSTANDING

Ask whether the accepted combination of related objects is valid as a whole.

— Make the shared-day conflict explicit

The DateRange behaviour from Step 7 can expose the factual relationship:

PYTHON

```

1  term_1_period = DateRange(
2      date(2026, 1, 1),
3      date(2026, 3, 28),
4  )
5
6  proposed_term_2_period = DateRange(
7      date(2026, 3, 28),
8      date(2026, 7, 28),
9  )
10
11  assert term_1_period.overlaps(
12      proposed_term_2_period
13  )

```

Both `DateRanges` are valid. Their relationship is the problem.

Because the boundaries are inclusive, 28 March belongs to both ranges. The existing `overlaps` behaviour correctly reports that fact.

But notice what `DateRange` does *not* say. It does not say:

These two periods are forbidden in Academic Year 2026.

That would be too much responsibility. `DateRange` knows only that two spans share a day. It does not know whether the spans belong to school Terms, holiday windows, examination periods, or some completely different part of the system.

CONTRAST

DateRange fact

These two inclusive ranges overlap on at least one day.

School calendar decision

These two numbered Terms may not occupy overlapping periods in the same `AcademicYear`.

The calculation is local to `DateRange`. The permission is not.

Term 2 cannot decide by looking harder at itself

Perhaps the rule belongs in `Term.reschedule`. That method is where the change happens, so the idea is initially attractive.

But Term 2 contains only its own information:

PYTHON

```

1  number = term_2.number
2  current_period = term_2.period

```

From those values alone, Term 2 cannot answer any of these questions:

- What period does Term 1 currently use?
- Has Term 3 already been scheduled?
- Does another Term already use TermNumber 2?
- Which `AcademicYear` owns these Terms?

- Would the proposed period place the year into an invalid overall order?

We could pass the missing information into the method:

PYTHON

```
1 term_2.reschedule(
2     proposed,
3     term_1,
4     term_3,
5     academic_year,
6 )
```

But the awkward signature is evidence. The business decision is no longer about Term 2 alone. We are trying to make one Entity coordinate objects that belong to a wider calendar.

KEY IDEA A rule cannot live meaningfully inside an object that lacks the information required to decide it. Passing the whole surrounding model into a child object is often evidence that the responsibility belongs at a wider level.

READING CUE

Missing information

No additional conditional inside Term 2 can manufacture information the Entity does not possess.

Return to the provisional AcademicYear

Back in Step 5, before we knew the DDD patterns involved, the rules forced us to sketch a wider concept:

PYTHON

```
1 @dataclass
2 class AcademicYear:
3     year: int
4     terms: list[Term]
```

At the time this was only a provisional representation. It stored the year and its Terms, but it did not protect them.

The shared-day conflict now tells us why this concept matters. AcademicYear is the smallest concept we have identified that can naturally answer questions such as:

What Terms are already accepted in Academic Year 2026, and would this proposed change leave them mutually consistent?

That question requires the year together with the Terms that belong to it.

We still have not decided the final design. The important discovery is the **information boundary** of the rule.

List the evidence needed before changing anything

Consider the request:

Reschedule Term 2 of Academic Year 2026 to 28 March–28 July.

Before the accepted state changes, the model needs enough information to evaluate at least the rules already discovered.

Question	Information required
Is the proposed period internally valid?	The proposed DateRange itself.
Is TermNumber 2 permitted?	The TermNumber value itself.
Does Term 2 already exist in this year?	The Terms belonging to AcademicYear 2026.
Does the proposal overlap another Term?	The proposed DateRange plus periods of the other scheduled Terms.
Does numbering still match chronology?	Term numbers and periods across the scheduled Terms.
Do the dates remain inside the 2026 teaching policy?	The AcademicYear value plus the proposed period.

Different rules need different amounts of context. The important point is that some of them require the same cluster of information: **the AcademicYear and its Terms**.

— The 28 March and 29 March examples reveal the boundary precisely

The best boundary examples change one fact at a time.

Term 1 remains:

PYTHON

```
1 term_1_period = DateRange(
2     date(2026, 1, 1),
3     date(2026, 3, 28),
4 )
```

First proposal:

PYTHON

```
1 starts_on_same_day = DateRange(
2     date(2026, 3, 28),
3     date(2026, 7, 28),
4 )
5
6 assert term_1_period.overlaps(
7     starts_on_same_day
8 )
```

Second proposal:

PYTHON

```
1 starts_next_day = DateRange(
2     date(2026, 3, 29),
3     date(2026, 7, 28),
4 )
5
6 assert not term_1_period.overlaps(
7     starts_next_day
8 )
```

The only changed fact is the opening date. The first proposal shares 28 March; the second does not.

This gives us a strong calendar-level expectation:

Example: One day changes the calendar result

Given Term 1 of Academic Year 2026 ends on 28 March.
When Term 2 is proposed to open on 28 March.
Then the calendar rejects the proposal because the Terms would share a day.

Example: The next day clears this conflict

Given Term 1 of Academic Year 2026 ends on 28 March.
When Term 2 is proposed to open on 29 March.
Then the proposal clears the shared-day rule, although the remaining AcademicYear rules must still be checked.

The test we need is no longer an isolated object test

A `DateRange` test can prove that overlap calculation works. A `Term` test can prove that rescheduling an already scheduled `Term` is a meaningful local transition. Neither test can prove that the complete `AcademicYear` remains valid.

We now need an executable example at the calendar level. Before deciding the final implementation, the test we want to be able to write looks conceptually like this:

PYTHON

```

1 def test_year_rejects_shared_day_between_terms():
2     year = AcademicYear(2026)
3     year.schedule_term(
4         TermNumber(1),
5         DateRange(
6             date(2026, 1, 1),
7             date(2026, 3, 28),
8         ),
9     )
10    year.schedule_term(
11        TermNumber(2),
12        DateRange(
13            date(2026, 5, 1),
14            date(2026, 7, 28),
15        ),
16    )
17
18    with pytest.raises(ValueError):
19        year.reschedule_term(
20            TermNumber(2),
21            DateRange(
22                date(2026, 3, 28),
23                date(2026, 7, 28),
24            ),
25        )

```

The methods in this test are not yet an implementation commitment. The test is a way of expressing the business outcome at the level where the required information appears to exist.

Most importantly, the example asks the *AcademicYear* to approve the change rather than mutating `Term 2` first and checking the damage afterwards.

READING CUE

One rule is not the whole decision

That final qualification matters. Passing one rule does not automatically prove that the whole proposed calendar is acceptable.

Rejection must preserve the accepted calendar

The requirement from BR-2.7 now becomes more important.

Suppose Term 2 currently has this accepted period:

PYTHON

```
1 original = DateRange(
2     date(2026, 5, 1),
3     date(2026, 7, 28),
4 )
```

A dangerous sequence would be:

PYTHON

```
1 term_2.reschedule(proposed)
2 check_the_rest_of_the_calendar()
```

If the second line discovers the overlap after Term 2 has already changed, the system must repair or roll back the invalid state.

A safer domain sequence is conceptually the reverse:

PYTHON

```
1 check_proposed_calendar_change()
2 apply_change_only_if_accepted()
```

Model decision: Validate before replacing accepted state

The exact implementation is still ahead of us, but the business requirement is already clear: **validate the proposed whole before replacing accepted state.**

A stronger test therefore includes the state-preservation assertion:

PYTHON

```
1 with pytest.raises(ValueError):
2     year.reschedule_term(
3         TermNumber(2),
4         proposed,
5     )
6
7 assert year.term(TermNumber(2)).period == original
```

The rejected request must leave the previously accepted Term 2 period untouched.

This problem is larger than overlap

The 28 March example is our clearest illustration, but it is not the only rule with this shape.

Consider what else can become invalid only when objects are viewed together:

- two individually valid Terms can use the same TermNumber in one year;
- two individually valid DateRanges can overlap;
- individually valid Terms can appear in an order that contradicts their numbers;

- a valid `DateRange` can belong to the wrong calendar year under SchoolGrid's teaching policy.

These are not four unrelated implementation accidents. They are evidence that a set of business rules governs the `AcademicYear` and its `Terms` as a coordinated whole.

Model decision: Do not push calendar-wide rules into child objects

`DateRange` protects `DateRange` validity. `TermNumber` protects the permitted numeric value. `Term` protects transitions it can decide from its own state. Rules that require several `Terms` or the owning year must be evaluated with the `AcademicYear` context before accepted state changes.

— A relationship alone is not yet the solution

It might be tempting to stop here and say:

`AcademicYear` contains a list of `Terms`, therefore the modelling problem is solved.

But storage is not protection. With the provisional public list, callers could still bypass every rule:

PYTHON

```
1 year.terms.append(  
2     Term(TermNumber(2))  
3 )
```

or obtain a `Term` and mutate it directly before the wider calendar has approved the change.

The real problem is therefore not merely that `AcademicYear` *references* `Terms`. The problem is deciding how the model should treat this group when rules must remain true across all of them.

That is the new design pressure.

— The question has changed again

At the start of Part II we asked what individual concepts meant. Then we asked how an `Entity` could protect its own state transitions.

We now have a different question:

When several related objects must obey rules together, what boundary should the model establish so that every supported change preserves those rules?

We have enough evidence to answer that question in the next step. Only now has the problem earned the next DDD pattern.

Try it: Find the smallest context that can decide the rule

For each business question below, identify the minimum information required to answer it. Then decide whether `DateRange`, `TermNumber`, `Term`, or `AcademicYear` has that information.

1. Does 20 May–1 May describe a valid date range?
 2. Is 4 a valid term number?
 3. Is this unscheduled `Term` allowed to be rescheduled?
 4. Does the proposed `Term 2` period overlap `Term 1`?
 5. Is there already another `Term 2` in `Academic Year 2026`?
 6. Would accepting the proposal preserve chronological ordering of all scheduled `Terms`?
 7. If a proposal is rejected, which previously accepted state must remain unchanged?
- Finally, explain why simply giving `Term` more methods does not solve questions whose answers depend on sibling `Terms` it cannot see.

NEXT QUESTION

If `AcademicYear` and its `Terms` must obey several business rules together, what DDD pattern defines the consistency boundary within which those rules must be protected?

Next: Step 12 – Aggregates: Protecting Consistency Boundaries

STEP 12

Aggregates: Protecting Consistency Boundaries

What you will discover

Derive an Aggregate from business rules that must hold across related objects, explain the meaning of a consistency boundary, and distinguish Aggregate membership from ordinary object relationships or broad real-world ownership.

Step 11 gave us the evidence before giving us the pattern.

Term 1 can be valid. Term 2 can be valid. Their DateRanges can each be valid. Their TermNumbers can each be valid. Yet the AcademicYear can still become invalid when those otherwise valid pieces are combined.

The clearest example remains the shared boundary day:

PYTHON

```
1 term_1_period = DateRange(  
2     date(2026, 1, 1),  
3     date(2026, 3, 28),  
4 )  
5  
6 proposed_term_2_period = DateRange(  
7     date(2026, 3, 28),  
8     date(2026, 7, 28),  
9 )
```

Both values satisfy the DateRange invariant. But they cannot be accepted as the periods of Term 1 and Term 2 in the same AcademicYear because both include 28 March.

The rule therefore does not belong to either DateRange in isolation. Nor can Term 2 decide it from its own state. The decision requires a wider group of objects to remain correct together.

The school does not approve a Term in isolation

The developer now asks the question that Step 11 made unavoidable.

DOMAIN WORKSHOP

What exactly has to stay correct together?

School administrator / Developer

DEV Developer

When you approve Term 2's dates, are you approving only Term 2?

SA School administrator

No. I need to know that the academic year still makes sense after the change.

DEV Developer

So I must compare the proposal with the other Terms already accepted for that year?

SA School administrator

Yes. Term 2 cannot be approved without considering Term 1 and, when it exists, Term 3.

DEV Developer

Would I also need every learner, teacher, subject, and fee record before approving the dates?

SA School administrator

No. Those are different concerns. For this decision I need the academic year and its term schedule.

That last answer is crucial. The consistency problem is wider than one Term, but it is not as wide as the whole school.

MODELLING

Boundary from rules

A modelling boundary should be justified by the rules that must hold together, not merely by the fact that objects are associated in the real world.

Model evolution

INITIAL INTERPRETATION

Group objects because they are related in the real world.

REVISED UNDERSTANDING

Group the smallest set of model objects whose business rules must remain consistent together when state changes.

Now the Aggregate concept has earned a name

Domain-Driven Design calls such a consistency boundary an **Aggregate**.

In SchoolGrid, the evidence points to the AcademicYear and its Terms.

Why?

Because several rules require that exact context:

- a TermNumber may occur only once within an AcademicYear;
- scheduled Term periods may not overlap;
- chronology must follow TermNumber order;

DDD CONCEPT

Aggregate

A cluster of domain objects treated as one consistency boundary for changes. The boundary is chosen so that the business invariants that must hold together can be protected together.

- under the current teaching policy, scheduled periods must remain inside the AcademicYear's calendar year;
- a rejected scheduling change must leave the previously accepted calendar unchanged.

No individual Term has enough information to guarantee all of these rules. The AcademicYear together with its Terms does.

DDD discovery: The SchoolGrid calendar Aggregate

For the calendar problem we are modelling, AcademicYear and its Terms form one consistency boundary. Changes to the schedule must preserve the rules that govern that group as a whole.

Misconception: Aggregate does not mean a list

The word *Aggregate* does not mean “a class containing a list.” It describes a modelling commitment about consistency.

A list of Terms was not enough

We already had this provisional class in Step 5:

PYTHON

```
1 @dataclass
2 class AcademicYear:
3     year: int
4     terms: list[Term]
```

At that stage it was merely a data structure. A caller could still do this:

PYTHON

```
1 year.terms.append(
2     Term(TermNumber(2))
3 )
```

or change a child Term independently:

PYTHON

```
1 term_2.reschedule(proposed_period)
```

Neither operation guarantees that the AcademicYear remains valid.

So the relationship

AcademicYear has Terms

is not what makes the model an Aggregate.

The stronger statement is:

These objects participate in business rules that must be preserved together whenever the accepted calendar changes.

KEY IDEA Aggregate boundaries are discovered from consistency rules, not from object containment alone. “Has many” is a structural relationship; “must remain valid together” is a consistency relationship.

The boundary answers a practical question

Imagine that Academic Year 2026 currently contains:

Term	Accepted period
Term 1	1 January–28 March 2026
Term 2	1 May–28 July 2026
Term 3	not yet scheduled

The school proposes moving Term 2 to 28 March–28 July.

To decide whether the change is acceptable, the model needs the proposed Term 2 period and the existing schedule in the same AcademicYear. It does not need the entire SchoolGrid database.

That gives the boundary a concrete purpose:

Inside this boundary are the objects whose state must be examined together before this calendar change can be accepted.

This is what **consistency boundary** means in this example.

DDD CONCEPT

Consistency boundary

The boundary inside which the model promises to preserve the relevant business invariants when a supported change is completed.

The boundary is not “everything connected to AcademicYear”

Real school concepts have many relationships. An AcademicYear may eventually be associated with learners, teachers, subjects, timetables, examinations, fees, and attendance records.

That does not mean all of those objects belong in this Aggregate.

DOMAIN WORKSHOP

Should the whole school be one Aggregate?

School administrator / Developer

DEV Developer

Everything in SchoolGrid belongs to the school somehow. Would it be safer to put learners, teachers, fees, subjects, attendance, and the calendar inside one big Aggregate?

SA School administrator

Would changing a teacher’s phone number require us to reapprove the Term dates?

DEV Developer

No. Those rules are unrelated.

SA School administrator

Then why should those records be part of the same consistency decision?

DEV Developer

They should not. The calendar boundary should contain only what the calendar rules need.

SA School administrator

Exactly. Being related to the same school is not enough.

Suppose a teacher's phone number changes. Does SchoolGrid need to load Term 1, Term 2, and Term 3 and revalidate calendar chronology before accepting the new phone number? No.

Suppose a learner's address changes. Must the academic calendar be locked and validated at the same time? Again, no.

Those changes do not participate in the calendar invariants we are protecting.

CONTRAST**Ordinary relationship**

Two concepts are associated or refer to one another, but their state does not have to be validated as one unit for this business rule.

Aggregate membership

Objects participate in invariants that must remain true together when the aggregate's accepted state changes.

This keeps the boundary deliberately small.

A gigantic "School Aggregate" containing every school-related object would make nearly every operation part of one consistency problem. That would be both a poor reflection of the business rules and an expensive implementation constraint.

Model decision: Keep the calendar boundary focused

For the current SchoolGrid model, the calendar Aggregate contains the AcademicYear and its Terms, including the Value Objects that form each Term's state. Learners, teachers, subjects, fees, attendance, and unrelated school concerns remain outside because the calendar invariants discovered so far do not require them to change consistently with the Term schedule.

MODELLING**Aggregate membership**

An object belongs inside an Aggregate because its state participates in invariants the Aggregate must protect, not simply because another object stores a reference to it.

Value Objects participate without creating new identity problems

Inside a Term we already use two Value Objects:

PYTHON

```
1 number: TermNumber
2 period: DateRange | None
```

They naturally participate in the Aggregate state because they describe the Term. But they do not need independent identity inside the boundary.

If Term 2 is rescheduled, the old DateRange is replaced by another DateRange value. If the TermNumber is 2, any independently constructed equal TermNumber(2) represents the same value.

The identity question remains centred on the Entities:

Academic Year 2026 contains Term 1, Term 2, and Term 3 as continuing domain things; their Value Objects describe their current state.

This is one reason DDD distinguishes Entities from Value Objects before Aggregate design. The Aggregate boundary may contain both, but they play different roles.

Move the test to the consistency boundary

Step 11 sketched the test we now need. We can make its purpose explicit:

PYTHON

```
1 def test_year_rejects_overlapping_term_periods():
2     year = AcademicYear(2026)
3     year.schedule_term(
4         TermNumber(1),
5         DateRange(
6             date(2026, 1, 1),
7             date(2026, 3, 28),
8         ),
9     )
10    year.schedule_term(
11        TermNumber(2),
12        DateRange(
13            date(2026, 5, 1),
14            date(2026, 7, 28),
15        ),
16    )
17
18    with pytest.raises(ValueError):
19        year.reschedule_term(
20            TermNumber(2),
21            DateRange(
22                date(2026, 3, 28),
23                date(2026, 7, 28),
24            ),
25        )
```

This is different from our earlier tests.

A `DateRange` test needs only one or two values. A `Term` test needs one Entity. This test deliberately needs several pieces of Aggregate state because the rule itself spans them.

That is not accidental test setup. It mirrors the business dependency of the rule.

TESTING

Aggregate-level example

A test belongs at Aggregate level when the business outcome cannot be decided from one child object in isolation.

The Aggregate test should also protect the old accepted state

Rejecting the bad proposal is not enough. The previously accepted calendar must remain intact.

So the test continues:

PYTHON

```
1 assert year.term(TermNumber(2)).period == DateRange(
2     date(2026, 5, 1),
3     date(2026, 7, 28),
4 )
```

The rejected 28 March proposal must not partially replace Term 2's period.

That gives us the essential change sequence inside this consistency boundary:

PYTHON

```

1 validate_proposed_state()
2 apply_change_only_if_valid()

```

The order matters.

If the model mutates Term 2 first and validates the AcademicYear afterwards, then an invalid Aggregate state exists between those operations. The code must then repair or roll back the damage.

If it evaluates the proposed state first, the accepted Aggregate remains valid when the proposal is rejected.

Example: Reject before changing accepted state

Given Academic Year 2026 contains Term 1 ending on 28 March and Term 2 beginning on 1 May.

When Term 2 is proposed to begin on 28 March.

Then the proposed calendar is rejected before Term 2's accepted period is replaced, so the existing 1 May–28 July period remains unchanged.

The Aggregate is defined by invariants, not by transaction technology

It is tempting to hear “consistency boundary” and immediately think about a SQL transaction, an ORM unit of work, or a database lock.

Those mechanisms may later help implement the design, but they are not where the boundary comes from.

We discovered the boundary before choosing a database because the business said:

Do not accept a Term change that leaves the AcademicYear inconsistent.

That is a domain requirement. Persistence technology comes later.

The smallest useful boundary is usually the better boundary

Why not include Academic Years 2025, 2026, and 2027 in one Aggregate?

Our current rules do not require that. The overlap and chronology rules concern Terms within the same AcademicYear. TermNumber uniqueness is also scoped to one year.

Term 2 of 2026 and Term 2 of 2027 may coexist perfectly well.

So forcing multiple academic years into one consistency boundary would add state without helping us enforce the rules we have discovered.

The same reasoning excludes the whole school.

READING CUE

Domain before technology

Likewise, an Aggregate is not automatically a microservice, a database table, or a folder in the source tree. It is first a domain modelling boundary around invariants that must be protected together.

Candidate boundary	Why keep or reject it?
One DateRange	Too small: cannot decide Term-to-Term rules.
One Term	Too small: cannot see sibling Terms.
One AcademicYear and its Terms	Fits the rules discovered so far.
Several AcademicYears together	Larger than required by the current invariants.
The entire school	Far larger than the calendar consistency problem.

Model decision: Choose the smallest useful boundary

Aggregate design is therefore not a contest to collect as many related objects as possible. The goal is to identify the smallest meaningful boundary that can protect the invariants that truly must hold together.

But a boundary drawn on paper does not protect anything

We now know that AcademicYear and its Terms should be treated as one consistency boundary. Yet our Python objects can still be misused.

A caller might keep a reference to Term 2 and invoke:

PYTHON

```
1 term_2.reschedule(
2     DateRange(
3         date(2026, 3, 28),
4         date(2026, 7, 28),
5     )
6 )
```

That direct call bypasses the wider validation we just said the Aggregate must perform.

So discovering the Aggregate boundary creates a new implementation question:

If outside code must not bypass the rules that protect the boundary, through which object should changes to the Aggregate be requested?

We have not solved that yet.

The answer will require one Entity inside the Aggregate to become the controlled entry point for operations that can affect Aggregate consistency.

Try it: Draw the boundary from rules, not nouns

Consider the SchoolGrid concepts AcademicYear, Term, DateRange, TermNumber, Learner, Teacher, Subject, FeeAccount, and AttendanceRecord. For each concept, ask whether the calendar rules discovered in Steps 1–11 require its state to be validated together with the Term schedule when a Term is scheduled or rescheduled.

Then answer these questions:

1. Why is one Term too small a consistency boundary for the calendar rules?
2. Why is the whole school too large a boundary for the same rules?
3. What evidence places AcademicYear and its Terms together?
4. Why does merely storing Terms in `year.terms` not protect the boundary?
5. What could go wrong if outside code keeps a Term reference and changes it directly?

NEXT QUESTION

If AcademicYear and its Terms form one Aggregate, which Entity should act as the controlled entry point so that outside code cannot bypass the rules by changing a child Term directly?

Next: Step 13 – The Aggregate Root

STEP 13

The Aggregate Root

What you will discover

Explain why an Aggregate needs a controlled entry point, identify AcademicYear as the Aggregate Root for the SchoolGrid calendar, distinguish reading child state from independently mutating child Entities, and explain what Python encapsulation can and cannot guarantee.

Step 12 gave us a boundary but not yet a safe way to use it.

We decided that AcademicYear and its Terms must be treated as one consistency boundary because several business rules span the whole group. A proposed change to Term 2 must be checked against the other Terms before the accepted calendar is changed.

But our current object model still lets a caller do exactly what that decision was supposed to prevent.

PYTHON

```
1 term_2 = year.term(TermNumber(2))
2
3 term_2.reschedule(
4     DateRange(
5         date(2026, 3, 28),
6         date(2026, 7, 28),
7     )
8 )
```

If Term 1 already includes 28 March, the calendar is now invalid. The caller changed the child Entity without asking the AcademicYear whether the proposed whole was acceptable.

The Aggregate exists conceptually, but its boundary can still be bypassed in code.

Model evolution

INITIAL INTERPRETATION

Know which objects must stay consistent together.

REVISED UNDERSTANDING

Control how outside code is allowed to change those objects.

The bypass appears in an ordinary use case

The developer shows the administrator two possible ways to implement the same request.

DOMAIN WORKSHOP

Who should receive the reschedule request?

School administrator / Developer

DEV Developer

Suppose the user asks to reschedule Term 2. I can first fetch Term 2 and tell the Term to change itself.

SA School administrator

Will Term 2 check Term 1 and Term 3 before changing?

DEV Developer

No. It cannot see them.

SA School administrator

Then you are bypassing the calendar check we just agreed was necessary.

DEV Developer

The alternative is to ask Academic Year 2026 to reschedule its Term 2.

SA School administrator

That sounds closer to the decision I make. I approve the change in the context of the year, not as an isolated edit to a Term.

The business action is still “reschedule Term 2,” but the object that has enough context to coordinate that action is AcademicYear.

READING CUE

Boundary versus entry point

The Aggregate boundary says what must remain consistent together. The Aggregate Root says where outside requests to change that boundary enter.

AcademicYear already has continuity of its own

Before naming the next DDD concept, we should confirm that AcademicYear is not merely a container object.

Academic Year 2026 may begin with no scheduled Terms, then gain Term 1, then Term 2, and later receive valid reschedules. Through all of those state changes the school still speaks about the same Academic Year 2026.

DOMAIN WORKSHOP

Does changing the schedule create a new Academic Year?

School administrator / Developer

DEV Developer

If I add Term 2 to Academic Year 2026, have I created a different academic year?

SA School administrator

No. It is still Academic Year 2026. We are configuring it.

DEV Developer

And if we later reschedule Term 2?

SA School administrator

Still the same academic year. Its calendar has changed, not its identity.

So AcademicYear is itself an Entity: it has continuing identity while its internal schedule evolves.

That makes it eligible to play a special role inside the Aggregate.

Now the Aggregate Root has earned a name

Domain-Driven Design designates one Entity in an Aggregate as the **Aggregate Root**.

For the SchoolGrid calendar Aggregate, that Entity is **AcademicYear**.

DDD discovery: AcademicYear is the Aggregate Root

Outside code requests scheduling and rescheduling through AcademicYear. The root uses the information inside the Aggregate to decide whether the proposed change is acceptable before allowing a child Term's state to change.

DDD CONCEPT

Aggregate Root

The Entity that represents the Aggregate to the outside and acts as the controlled entry point for operations that may affect Aggregate consistency.

This gives us a more precise model:

Concept	Role in the calendar model
AcademicYear	Aggregate Root; receives supported calendar changes and coordinates consistency.
Term	Child Entity; has identity within the AcademicYear and owns local Term behaviour.
TermNumber	Value Object; represents the permitted numbered position.
DateRange	Value Object; represents and validates a period.

Misconception: Aggregate and Aggregate Root are not synonyms

The Aggregate is the entire consistency boundary. The Aggregate Root is one Entity inside that boundary that represents it to outside code. AcademicYear is the root; AcademicYear together with its Terms and their values is the Aggregate.

Move the supported change to the root

The direct child call from the beginning of the step was:

PYTHON

```
1 term_2.reschedule(proposed_period)
```

The supported operation should instead express the full business context:

PYTHON

```

1 year.reschedule_term(
2     TermNumber(2),
3     proposed_period,
4 )

```

That is not merely a longer method call. The receiver has changed from the child Entity to the object that can see the consistency boundary.

The first implementation can focus on the overlap rule that exposed the need for this design:

PYTHON

```

1 def reschedule_term(self, number, new_period):
2     term = self._term(number)
3
4     for other in self._terms:
5         if other.number == number:
6             continue
7         if not other.is_scheduled:
8             continue
9         if other.period.overlaps(new_period):
10            raise ValueError("term periods overlap")
11
12     term._reschedule(new_period)

```

The method first inspects the sibling Terms. Only after the proposal clears that check does it ask the selected Term to replace its period.

The underscore on `_reschedule` is deliberate. The state-changing method still belongs to Term because Term owns that local state, but it is no longer advertised as the operation outside callers should invoke directly.

READING CUE

Coordination, not ownership of every detail

The root does not absorb all child behaviour. It coordinates changes that need Aggregate-wide information, while child objects still own behaviour that belongs to them.

The root validates before the child mutates

The ordering of the previous method is the important part:

PYTHON

```

1 check_the_proposal()
2 term._reschedule(new_period)

```

not:

PYTHON

```

1 term._reschedule(new_period)
2 check_the_calendar_afterwards()

```

The second sequence temporarily creates an invalid accepted state and then hopes to repair it if validation fails.

The first sequence treats the new DateRange as a proposal until the root has enough evidence to accept it.

Example: The root rejects before mutation

Given Term 1 ends on 28 March and Term 2 currently runs from 1 May to 28 July.

When AcademicYear is asked to reschedule Term 2 so that it begins on 28 March.

Then AcademicYear detects the overlap before changing Term 2, rejects the request, and Term 2 remains scheduled from 1 May to 28 July.

The corresponding test now exercises the public Aggregate operation:

PYTHON

```
1 def test_rejected_reschedule_preserves_term_2():
2     year = configured_2026_calendar()
3     original = year.period_for(TermNumber(2))
4
5     with pytest.raises(ValueError):
6         year.reschedule_term(
7             TermNumber(2),
8             DateRange(
9                 date(2026, 3, 28),
10                date(2026, 7, 28),
11            ),
12        )
13
14     assert year.period_for(TermNumber(2)) == original
```

Notice that the assertion asks the root for the observed period rather than reaching through the root to mutate or inspect a public child field.

Reading child information is different from controlling child mutation

An Aggregate Root does not mean outside code must become blind to everything inside the Aggregate. A caller may need to display Term 2's dates or ask whether Term 3 has been scheduled.

What matters is that observation does not create an unsupported path for changing Aggregate state.

One small root query can expose the information needed by the caller:

PYTHON

```
1 def period_for(self, number):
2     return self._term(number).period
```

Because DateRange is immutable, returning that value does not allow the caller to mutate Term 2 indirectly.

We can expose other questions in the same way when a real use case needs them. There is no requirement to publish every child object merely because it exists.

DOMAIN WORKSHOP

Can the UI still display the Terms?

School administrator / Developer

DEV

Developer

If AcademicYear is the only entry point, does that mean the screen cannot show Term 2's current dates?

SA

School administrator

Of course it can show them. The restriction is about changing the calendar without the year checking it.

DEV

Developer

So reading information and independently changing a child are different concerns?

SA

School administrator

Exactly. Showing the accepted calendar should not create a back door around the calendar rules.

READING CUE

Read is not write

The root may expose information from child objects without granting an independent mutation path to those children. The API should reflect that difference.

A tuple protects a collection less than it first appears

A common Python idea is to keep the internal list private and return a tuple:

```
PYTHON
1 @property
2 def terms(self):
3     return tuple(self._terms)
```

This prevents callers from doing this to the returned collection:

```
PYTHON
1 year.terms.append(new_term)
```

because a tuple has no append method.
That is useful, but it does *not* make the Aggregate immutable.
The tuple still contains references to the same Term objects:

```
PYTHON
1 term = year.terms[1]
2 term._reschedule(proposed_period)
```

A read-only collection is therefore not the same thing as read-only child Entities.
This distinction is easy to miss because two different kinds of mutation are involved:

Mutation	Does a tuple prevent it?
Add or remove an element from the returned collection	Yes. The tuple structure is immutable.
Change state inside a mutable Term referenced by the tuple	No. The Term is still the same mutable object.

So returning a tuple may be part of a careful API, but it is not sufficient proof that callers cannot bypass the Aggregate Root.

PYTHON

Reference leak

A mutable object can still be changed through any reference that reaches it. Wrapping references in an immutable tuple prevents changing the tuple structure, not the referenced objects themselves.

— Python gives us conventions, not absolute privacy

Languages differ in how strongly they can hide internal members. Python's leading underscore communicates that a member is non-public by convention:

PYTHON

```
1 term._reschedule(new_period)
```

A determined caller can still invoke it.

Double-underscore name mangling can make accidental access harder, but it is not a security boundary either. Python deliberately relies heavily on convention and API discipline.

This does not make the Aggregate Root idea meaningless. It changes what good implementation discipline looks like:

- outside use cases call public operations on `AcademicYear`;
- child mutation methods are treated as internal implementation details;
- tests exercise the public root operations for `Aggregate` behaviour;
- modules and naming make unsupported paths visibly exceptional rather than convenient;
- code review treats direct child mutation from outside the `Aggregate` as a boundary violation.

The goal is not to make malicious Python code impossible. The goal is to make the correct domain path obvious and the bypass path clearly unsupported.

Misconception: Private syntax is not the Aggregate Root

Adding underscores does not create a DDD boundary by itself. `AcademicYear` is the `Aggregate Root` because the domain requires calendar-wide consistency and the model routes supported changes through the `Entity` that can protect it. The underscore merely helps the Python implementation communicate that decision.

PYTHON

Encapsulation

Designing an object's public interface so callers use supported operations while implementation details remain internal. In Python, underscores communicate this boundary but do not make access physically impossible.

— The root should not become a god object

Once `AcademicYear` becomes the entry point, another bad design becomes tempting: move every method from every child into `AcademicYear`.

That would solve the wrong problem.

`Term` still knows whether it is currently scheduled. `DateRange` still knows whether two periods overlap. `TermNumber` still protects the values 1, 2, and 3. The root coordinates these concepts when a decision requires their combined information.

For example, the root can ask a `DateRange` factual question:

PYTHON

```
1 other.period.overlaps(new_period)
```

and then use the answer to enforce the `AcademicYear` policy.

It does not need to reimplement `DateRange` arithmetic inside `AcademicYear`.

KEY IDEA An Aggregate Root coordinates consistency; it does not replace the behaviour of every object inside the Aggregate. Good Aggregate design preserves responsibility at the smallest concept that has enough information to own it.

— The root also protects how child Entities are created

The same boundary applies when a new Term is scheduled.

A caller should not create a Term and append it directly to some public list:

PYTHON

```
1 year.terms.append(
2     Term(TermNumber(2))
3 )
```

Instead, the supported action enters through the root:

PYTHON

```
1 year.schedule_term(
2     TermNumber(2),
3     DateRange(
4         date(2026, 5, 1),
5         date(2026, 7, 28),
6     ),
7 )
```

The root can then decide whether that numbered Term may be added to the current AcademicYear before changing the internal collection.

We are deliberately not implementing every scheduling rule in this step. The important architectural change is that **there is now one supported place where those rules can be enforced consistently**.

Model decision: External changes enter through AcademicYear

AcademicYear is the public entry point for scheduling and rescheduling Terms in its calendar Aggregate. Term continues to own local state behaviour, but direct child mutation is treated as internal to the Aggregate. The root validates the proposal using Aggregate context before allowing accepted child state to change.

— What the root has solved—and what it has not

We now have a credible answer to the bypass problem.

Outside code no longer needs to know how Term 2 mutates itself. It asks Academic Year 2026 to perform the business operation. AcademicYear can inspect its Terms, reject an invalid proposal before mutation, and then delegate the local state change to the selected child.

But a new question immediately follows.

Our example checked overlap. The school has already discovered several other calendar rules: duplicate TermNumbers, chronological ordering, the current calendar-year containment policy, progressive scheduling, and preservation of accepted state after rejection.

If AcademicYear is now responsible for protecting the Aggregate boundary, we must state precisely

which conditions must always remain true after a supported operation completes.

That is the next modelling task.

Try it: Find the real entry point

Consider each line below and decide whether it should be part of the supported outside API of the calendar Aggregate. Explain what rule could be bypassed if the wrong object receives the change.

1. `year.reschedule_term(TermNumber(2), proposed)`
2. `term_2._reschedule(proposed)` from application code
3. `year.period_for(TermNumber(2))`
4. returning `tuple(year._terms)` to a caller
5. appending a newly constructed Term directly to the internal collection

Then explain why a tuple can protect collection structure without protecting the mutable state of every object referenced inside it.

NEXT QUESTION

Now that every supported calendar change enters through `AcademicYear`, exactly which conditions must the root guarantee are true after each accepted operation—and which incomplete states are still valid while the year is being configured?

Next: Step 14 – Aggregate Invariants

STEP 14

Aggregate Invariants

What you will discover

State the SchoolGrid calendar invariants precisely, distinguish Aggregate consistency from life-cycle completeness, and use examples to show how AcademicYear can remain valid while it is being configured progressively.

Step 13 gave AcademicYear a clear responsibility: every supported scheduling change enters through the Aggregate Root so that calendar-wide rules cannot be bypassed by changing a child Term directly.

That immediately creates a harder question.

What, exactly, must AcademicYear guarantee before it accepts a change?

We already have the raw material. Steps 1 and 2 discovered rules about permitted TermNumbers, duplicate Terms, calendar-year containment, chronology, shared days, and rejected changes. The Aggregate work in Steps 11–13 showed that several of those rules need the whole AcademicYear context.

Now we must turn those agreements into precise conditions that define an acceptable Aggregate state.

Model evolution

INITIAL INTERPRETATION

Route every supported change through the Aggregate Root.

REVISED UNDERSTANDING

State the conditions the root must preserve after every accepted change.

Start with a draft year, not an imaginary finished year

The administrator begins configuring Academic Year 2026 before every Term date is known.

She may first enter only Term 2:

PYTHON

```
1 year = AcademicYear(2026)
2
3 year.schedule_term(
4     TermNumber(2),
5     DateRange(
6         date(2026, 5, 1),
7         date(2026, 7, 28),
8     ),
9 )
```

At first glance this can look suspicious. There is no Term 1 yet. There is no Term 3 either.

But BR-2.4 already told us that SchoolGrid allows the year to be configured progressively.

DOMAIN WORKSHOP

Is Term 2 allowed to exist before Term 1 is entered?

School administrator / Developer

DEV Developer

If you enter Term 2 first, have we violated the rule that Term 1 comes before Term 2?

SA School administrator

No. I have only entered the information in a different order. I have not said that Term 2 occurs before Term 1 in the calendar.

DEV Developer

So data-entry order and calendar order are different things?

SA School administrator

Exactly. When both Terms have dates, their dates must follow their numbers. But I do not need to type them into SchoolGrid in numerical order.

That distinction determines how strict the Aggregate is allowed to be.

An AcademicYear containing only Term 2 can therefore be an acceptable *intermediate state*.

An invariant is not the same thing as completeness

We first met the idea of an invariant with DateRange in Step 7. There the condition was local: a closing date must never precede its opening date.

At Aggregate level, an **invariant** is a condition that must remain true for the accepted Aggregate state after every supported operation completes.

That wording matters because a different kind of requirement may apply only at a particular lifecycle stage.

Imagine the school later says:

Do not publish the academic calendar until all three Terms have been scheduled.

That could be a perfectly reasonable publication rule. But it would not mean that an AcademicYear containing only Term 1 is internally inconsistent while the administrator is still preparing it.

CONTRAST

Aggregate invariant

Must hold after every accepted Aggregate change.
Example: two scheduled Terms in the same year may not contradict their number order.

Lifecycle completeness rule

Applies when performing a particular transition such as publishing. A draft may remain incomplete before that transition.

We have not yet discovered a SchoolGrid publication requirement, so we will not invent one. The example simply shows why **incomplete** and **invalid** must not be treated as synonyms.

READING CUE

Configuration order versus calendar order

Term 2 may be entered before Term 1. The invariant constrains the dates of Terms that are present; it does not prescribe the order in which administrators must enter them.

DDD CONCEPT

Aggregate invariant

A business condition that must hold for every accepted state of an Aggregate, including valid intermediate states while the Aggregate evolves.

Misconception: A valid Aggregate need not be a finished Aggregate

Progressive configuration means a draft AcademicYear can be valid with only some Terms scheduled. Requiring all three Terms after every operation would contradict the business evidence from Step 2.

Invariant 1: a TermNumber is unique within the year

The first Aggregate rule is already familiar from BR-2.3.

If Academic Year 2026 already contains Term 2, another request must not create a second Term 2.

A test makes both rejection and state preservation visible:

PYTHON

```
1 def test_year_rejects_a_second_term_2():
2     year = AcademicYear(2026)
3     original = DateRange(
4         date(2026, 5, 1),
5         date(2026, 7, 28),
6     )
7     year.schedule_term(TermNumber(2), original)
8
9     with pytest.raises(ValueError):
10        year.schedule_term(
11            TermNumber(2),
12            DateRange(
13                date(2026, 5, 10),
14                date(2026, 7, 28),
15            ),
16        )
17
18    assert year.period_for(TermNumber(2)) == original
```

The proposed DateRange is valid. TermNumber 2 is also valid. The problem is the combination: the Aggregate would contain two child Entities claiming the same identity within the year.

The root can detect that before creating the new child:

PYTHON

```
1 def _ensure_number_available(self, number):
2     if any(term.number == number for term in self._terms):
3         raise ValueError("term number already exists")
```

This invariant does not say every permitted number must already exist. It says that a number which *does* identify a Term in this year may identify at most one such Term.

Invariant 2: scheduled dates remain inside the named year

The teaching policy from BR-2.5 says that Term dates for Academic Year 2026 must remain in calendar year 2026.

A DateRange such as this is internally valid:

READING CUE**Scoped uniqueness**

TermNumber 2 may exist many times as a Value Object. What must be unique is the Term identified by that number inside one AcademicYear.

PYTHON

```

1 proposed = DateRange(
2     date(2026, 9, 1),
3     date(2027, 1, 15),
4 )

```

Its closing date comes after its opening date. `DateRange` has no reason to reject it.

`AcademicYear` does:

PYTHON

```

1 def test_year_rejects_term_crossing_into_2027():
2     year = AcademicYear(2026)
3
4     with pytest.raises(ValueError):
5         year.schedule_term(
6             TermNumber(3),
7             DateRange(
8                 date(2026, 9, 1),
9                 date(2027, 1, 15),
10            ),
11        )

```

The root has the piece of information `DateRange` lacks: which `AcademicYear` the period is being proposed for.

The corresponding check is small:

PYTHON

```

1 def _ensure_within_year(self, period):
2     if period.opening_date.year != self._year:
3         raise ValueError("opening date is outside academic year")
4     if period.closing_date.year != self._year:
5         raise ValueError("closing date is outside academic year")

```

This is a SchoolGrid teaching policy, not a universal truth about academic years. A different school model could legitimately allow a year to cross a calendar-year boundary.

MODELLING

Contextual validity

A value may be valid by itself yet unacceptable in a particular context. The `DateRange` is chronological; `AcademicYear 2026` still rejects it because the closing date belongs to 2027.

Invariant 3: Term numbers determine chronological order

Now return to BR-2.6.

Suppose Term 2 is already scheduled from 1 May to 28 July. Later the administrator enters Term 1 as 1 September to 28 November:

PYTHON

```

1 term_2_period = DateRange(
2     date(2026, 5, 1),
3     date(2026, 7, 28),
4 )
5
6 proposed_term_1 = DateRange(
7     date(2026, 9, 1),
8     date(2026, 11, 28),
9 )

```

The two ranges do not overlap. Yet the calendar is still wrong: Term 1 would occur after Term 2.

DOMAIN WORKSHOP

Is non-overlap enough?

School administrator / Developer

DEV Developer

Term 2 ends in July and the proposed Term 1 begins in September. They do not share a day. Can I accept both?

SA School administrator

No. Term 1 must occur before Term 2.

DEV Developer

So “no overlap” is necessary, but it does not capture the whole rule?

SA School administrator

Correct. The numbers describe the order of the Terms, not merely labels attached to unrelated periods.

That gives us the stronger pairwise condition:

For any two scheduled Terms in one AcademicYear, the lower-numbered Term must close strictly before the higher-numbered Term opens.

Because the comparison is strict, the condition also prevents the two Terms from sharing a boundary day.

A helper can express the rule relative to a proposed period:

PYTHON

```
1 def _ensure_fits_schedule(self, number, period):
2     for other in self._terms:
3         if not other.is_scheduled or other.number == number:
4             continue
5
6         if number < other.number:
7             if period.closing_date >= other.period.opening_date:
8                 raise ValueError("term order conflict")
9         else:
10            if other.period.closing_date >= period.opening_date:
11                raise ValueError("term order conflict")
```

The helper does not care which Term was entered first. It compares the *numbered positions* and the *calendar dates* that are currently known.

— The shared-day boundary still matters

The strict comparison explains the boundary example that has followed the book since Step 1.

Term 1 ends on 28 March:

READING CUE

One condition does two jobs

If every lower-numbered scheduled Term closes strictly before every higher-numbered scheduled Term opens, the calendar has both the required number order and no overlap between Terms.

PYTHON

```

1 term_1 = DateRange(
2     date(2026, 1, 1),
3     date(2026, 3, 28),
4 )

```

A Term 2 opening on the same day is rejected:

PYTHON

```

1 shared_day = DateRange(
2     date(2026, 3, 28),
3     date(2026, 7, 28),
4 )
5
6 assert term_1.closing_date >= shared_day.opening_date

```

A Term 2 opening the following day clears that particular rule:

PYTHON

```

1 next_day = DateRange(
2     date(2026, 3, 29),
3     date(2026, 7, 28),
4 )
5
6 assert term_1.closing_date < next_day.opening_date

```

This is not merely a DateRange fact. The Aggregate interprets the relationship in light of TermNumber order.

— Gaps are allowed because no rule forbids them

The current 2026 example actually contains a gap between Term 1 and Term 2:

Term	Period
Term 1	1 January–28 March
Term 2	1 May–28 July

There are unscheduled days between 29 March and 30 April. Nothing in the rules says the Terms must touch each other with no gap.

A developer could easily over-generalise the strict ordering rule into a false adjacency rule. The following expression would be too strict for the rules we actually discovered:

PYTHON

```

1 next_term.opening_date == previous_term.closing_date + one_day

```

SchoolGrid has no evidence for that requirement.

DOMAIN WORKSHOP

Must every Term start the day after the previous one closes?

School administrator / Developer

DEV Developer

To keep the calendar orderly, should Term 2 always open immediately after Term 1 closes?

SA School administrator

No. We can have holidays and other gaps between Terms.

DEV Developer

So the rule is “later,” not “the very next day”?

SA School administrator

Exactly. Do not turn an example into a tighter rule than the school gave you.

DISCOVERY TOOL

Over-constraint

A model is over-constrained when it rejects states the business actually permits. Adding unsupported restrictions is as much a modelling error as failing to enforce a real rule.

The invariant therefore allows gaps.

Partial schedules are also allowed

Now consider a year with only Term 2 scheduled:

PYTHON

```
1 year = AcademicYear(2026)
2 year.schedule_term(
3     TermNumber(2),
4     DateRange(
5         date(2026, 5, 1),
6         date(2026, 7, 28),
7     ),
8 )
```

There is no Term 1 period to compare with and no Term 3 period to compare with. The pairwise chronology invariant has nothing to reject.

Later, Term 1 can be proposed. At that moment the Aggregate finally has enough information to compare the two:

PYTHON

```
1 year.schedule_term(
2     TermNumber(1),
3     DateRange(
4         date(2026, 1, 1),
5         date(2026, 3, 28),
6     ),
7 )
```

The year remains valid because the newly known pair satisfies the rule.

A different Term 1 proposal would fail at the moment it creates contradictory information:

PYTHON

```

1  DateRange(
2      date(2026, 9, 1),
3      date(2026, 11, 28),
4  )

```

This is a subtle but important model: incomplete information is permitted, but contradictory information is not.

READING CUE**Validate what is known**

Progressive configuration does not suspend invariants. The root enforces every invariant that can be evaluated from the state currently known, without inventing missing information.

Bring the invariant checks into the root operation

We can now see the structure of `schedule_term` more clearly.

Before mutating the Aggregate, the root checks the rules for which the proposal provides enough information:

PYTHON

```

1  def schedule_term(self, number, period):
2      self._ensure_number_available(number)
3      self._ensure_within_year(period)
4      self._ensure_fits_schedule(number, period)
5
6      term = Term(number)
7      term._schedule(period)
8      self._terms.append(term)

```

The order is deliberate. All Aggregate-level rejection happens before the new Term becomes accepted state.

Rescheduling follows the same shape:

PYTHON

```

1  def reschedule_term(self, number, new_period):
2      term = self._term(number)
3      self._ensure_within_year(new_period)
4      self._ensure_fits_schedule(number, new_period)
5
6      term._reschedule(new_period)

```

The difference between scheduling and rescheduling remains meaningful, but the same Aggregate invariants protect the proposed period in both operations.

Model decision: Every accepted root operation preserves the calendar invariants

AcademicYear rejects duplicate Term identities, dates outside the configured calendar year, and Term periods that contradict TermNumber chronology. It allows gaps and partially configured years because no discovered invariant forbids them. The root validates proposed Aggregate state before applying the mutation.

A rule-to-example matrix keeps the model honest

At this point the Aggregate behaviour is easier to audit if each rule is paired with an example near its boundary.

Example	What it probes	Outcome
Add another Term 2	Scoped Term identity must be unique	Reject
Term 3 closes 15 Jan 2027	Scheduled dates remain in 2026	Reject
Term 1 Sep–Nov, Term 2 May–Jul	Number chronology, despite no overlap	Reject
Term 1 closes 28 Mar, Term 2 opens 28 Mar	Inclusive shared day	Reject
Term 1 closes 28 Mar, Term 2 opens 29 Mar	Strict ordering boundary	Clears this rule
Term 1 closes 28 Mar, Term 2 opens 1 May	Gaps are permitted	Acceptable
Only Term 2 is scheduled	Progressive configuration	Acceptable
Term 2 entered before Term 1	Data-entry order is not calendar order	Acceptable until contradictory dates appear

The last two rows are especially important. Without them, a test suite could accidentally encode the false requirement that AcademicYear must always contain a complete three-Term calendar.

— One failed rule must not damage another accepted state

Every invariant check also inherits the preservation rule from BR-2.7.

Suppose Term 2 is currently accepted as 1 May–28 July. A reschedule to 28 March–28 July violates chronology with Term 1.

The root must reject the proposal and leave this observation unchanged:

PYTHON

```
1 assert year.period_for(TermNumber(2)) == DateRange(
2     date(2026, 5, 1),
3     date(2026, 7, 28),
4 )
```

The same applies to a duplicate TermNumber or a cross-year proposal. Rejection is not merely an exception being raised; it is the guarantee that the previously accepted Aggregate state survives the failed operation.

DOMAIN WORKSHOP

What should remain after a rejected change?

School administrator / Developer

DEV Developer

If a proposed Term 2 date breaks one of these rules, is it enough that SchoolGrid displays an error?

SA School administrator

No. When I return to the calendar, I expect the last accepted dates still to be there.

DEV Developer

So rejection has an observable state guarantee, not just an error message?



School administrator

Exactly. A failed request must not leave half of its change behind.

READING CUE

Failure has a postcondition too

For a rejected Aggregate operation, an important postcondition is that the previously accepted state remains intact.

Do not confuse an invariant with every business rule in the system

Not every rule involving an AcademicYear must become an Aggregate invariant.

Suppose a future requirement says:

The headteacher must approve the calendar before parents receive it.

That is a business rule, but it does not automatically mean that every in-memory AcademicYear without headteacher approval is invalid. It may instead govern a specific use case such as publication.

Likewise, a rule that only administrators may reschedule Terms concerns *authorization*. It may be enforced before the domain operation is invoked rather than encoded as structural consistency of the Aggregate itself.

CONTRAST

Aggregate invariant

Defines which Aggregate states are acceptable after supported domain changes.

Other business rule

May govern who may request an operation, when a lifecycle transition is allowed, or what happens outside the Aggregate.

The Aggregate should protect the consistency rules that define its valid state, not become a dumping ground for every policy that mentions AcademicYear.

The invariant is about the accepted whole

We can now summarise the current SchoolGrid calendar invariant more precisely.

For Academic Year 2026, every accepted state must satisfy all of the following for the Terms that currently exist:

- each Term has a permitted TermNumber, already guaranteed by TermNumber;
- no two Terms in the year have the same TermNumber;
- every scheduled DateRange belongs to calendar year 2026 under the current teaching policy;
- for any two scheduled Terms, the lower-numbered Term closes strictly before the higher-numbered Term opens;
- gaps are allowed;
- Terms may be missing while the year is being configured progressively.

This formulation does not say “exactly three Terms must always exist.” It does not say Terms must be entered in numerical order. It does not say one Term must begin immediately after the previous one closes.

Those omissions are intentional. A precise model protects the rules that the business actually gave us and leaves unsupported restrictions out.

KEY IDEA An Aggregate invariant should be strong enough to exclude every state the business has declared inconsistent, but no stronger. Progressive configuration means SchoolGrid must preserve consistency without confusing incompleteness with invalidity.

— The next problem is not another rule

At this point AcademicYear has a coherent set of conditions to protect, and the root can validate proposed changes before mutating its child Terms.

In memory, that gives us an important guarantee: a rejected operation need not leave a partially changed Aggregate.

But SchoolGrid will eventually save AcademicYear somewhere durable. That raises a new class of questions.

What if the program changes the Aggregate correctly in memory but the database save fails halfway through? What if two administrators both load Academic Year 2026, make different individually valid changes, and save them without noticing that one was based on stale state?

Those questions are related to the consistency boundary, but they are not solved merely by writing another invariant.

Try it: Invariant or lifecycle rule?

For each statement, decide whether it describes an invariant of every accepted draft AcademicYear, a possible lifecycle/use-case rule, or a rule we have no evidence for. Explain the distinction.

1. No two Terms in one AcademicYear share the same TermNumber.
2. Every scheduled Term in Academic Year 2026 has dates in 2026.
3. All three Terms must be scheduled before any draft can be saved.
4. All three Terms must be scheduled before a future “publish calendar” operation succeeds.
5. Term 2 may be entered before Term 1.
6. Two scheduled Terms may not contradict their numeric order.
7. Term 2 must start exactly one day after Term 1 closes.

Then explain why a requirement can be important to the business without necessarily being an Aggregate invariant.

NEXT QUESTION

If the Aggregate’s invariants must remain true through every accepted change, what does that imply about applying and saving a change as one atomic unit—and what additional problems appear when persistence failures or concurrent editors enter the picture?

Next: Step 15 – Transactional Consistency and Why Aggregates Matter

STEP 15

Transactional Consistency and Why Aggregates Matter

The SchoolGrid situation

A rejected reschedule must not partly change the calendar. Later, two administrators may also try to save different changes at the same time.

Ideas to develop

First reason about validating a proposed state before mutation. Then distinguish this in-memory guarantee from durable commit and concurrency control. Introduce a version check as one possible later persistence mechanism; an aggregate alone does not prevent a stale save from overwriting another user's work.

Example: The example to build on

Given Term 2 has an accepted period.

When a reschedule would overlap Term 1.

Then the operation is rejected and the original schedule is preserved.

Testing checkpoint

Start with a state-preservation domain test. Reserve commit failure and competing-save tests for a real persistence boundary.

Try it: Explain before implementing

Can two individually valid changes produce an invalid combined schedule when saved without concurrency protection?

READING CUE

What you will discover

Distinguish an atomic domain operation, a database transaction, and concurrent-update protection.

STEP 16

Domain Services: Behaviour That Does Not Naturally Belong to One Object

READING CUE

What you will discover

Identify a genuine domain operation without inventing a service for every rule.

The SchoolGrid situation

As an optional extension, two schools want to find shared teaching dates for an inter-school activity by comparing their independently managed calendars.

Ideas to develop

Explore whether the comparison belongs to either calendar or to a stateless domain operation that accepts both. Keep repository loading outside that calculation. Compare a function, a policy object, and a service class. The core one-school scheduling model does not require a domain service simply to complete a pattern checklist.

Example: The example to build on

Given two valid calendars have been supplied for comparison.

When the shared-teaching-date calculation is requested.

Then it returns dates satisfying both calendars without changing either one.

Testing checkpoint

Test common dates, no common dates, and unchanged input calendars using ordinary domain objects.

Try it: Explain before implementing

What would distinguish this domain calculation from the application work of loading the calendars?

STEP 17

Factories: Creating Valid Domain Objects

The SchoolGrid situation

Creating a term is not merely allocating an object: its number must be available and its dates acceptable within the year.

Ideas to develop

Compare simple DateRange construction with creating a Term through its AcademicYear. Introduce factory behaviour as coordinated creation, not necessarily a separate class. Consider a future three-term calendar template only as an optional construction problem. Keep reconstitution from storage distinct from a new business creation event.

Example: The example to build on

Given Term 1 already exists in the year.

When creation of another Term 1 is requested.

Then creation is refused rather than returning an invalid new child for callers to repair.

Testing checkpoint

Test valid creation and rejection before insertion; check that failed construction leaves the owner unchanged.

Try it: Explain before implementing

When would a separate factory add clarity beyond a constructor or root operation?

READING CUE

What you will discover

Distinguish constructors, factory methods, and dedicated factories, and justify their use.

STEP 18

Domain Events: Recording That Something Important Happened

READING CUE

What you will discover

Distinguish commands from past business facts and recording from reliable external delivery.

The SchoolGrid situation

After a successful reschedule, the school may want another part of the system to react. This is an optional extension of the calendar story.

Ideas to develop

Explore `AcademicYearCreated`, `TermScheduled`, and `TermRescheduled` as past-tense facts with sufficient identity and relevant data. Keep event recording inside the successful business operation separate from notification delivery. Introduce transactional outbox and idempotency only as later reliability concerns, not prerequisites for understanding an event.

Example: The example to build on

Given a valid reschedule changes Term 2's period.

When the operation completes successfully.

Then a `TermRescheduled` fact can be recorded; a rejected change must not announce success.

Testing checkpoint

Test event meaning on success and absence on rejection. Defer delivery, retry, and transaction tests to integration.

Try it: Explain before implementing

Why is publishing an external message before the database commit risky?

STEP 19

Repositories: Thinking in Aggregates, Not Tables

The SchoolGrid situation

An administrator returns tomorrow to change the 2026 calendar. The application must recover the same academic year and its scheduled terms.

Ideas to develop

Introduce a collection-like repository contract for `AcademicYear`, not separate independent write repositories for its child `Terms`. Describe getting an aggregate and recording its accepted state without selecting a database technology. Make clear that the repository abstraction and its storage implementation are different concerns.

Example: The example to build on

Given the 2026 academic year has been persisted.

When the use case retrieves that year.

Then it receives the domain state needed to evaluate a later scheduling request.

Testing checkpoint

Plan a reusable retrieval-and-save contract, first supported by an in-memory fake and later checked against actual storage.

Try it: Explain before implementing

Why might a repository return `AcademicYear` rather than a row for Term 2 alone?

READING CUE

What you will discover

Explain retrieval and persistence in terms of aggregate roots and business identity.

STEP 20

What Repositories Should and Should Not Do

READING CUE

What you will discover

Distinguish aggregate repositories, data-access objects, DTOs, and read projections.

The SchoolGrid situation

A calendar screen needs only term numbers and dates. Must it load every aggregate and use the same path as a rescheduling command?

Ideas to develop

Compare an aggregate-oriented write repository with a storage-oriented DAO and a read query that projects data. Explain why children do not automatically need repositories and why a DTO result is not an aggregate. Keep query design flexible; do not turn a useful repository convention into a universal rule for every read.

Example: The example to build on

Given the user only wants a calendar summary.

When a read query selects term numbers and dates.

Then it can return a projection without pretending that the projection can enforce scheduling rules.

Testing checkpoint

Separate repository behaviour tests from projection-shape tests and mutation-rule tests.

Try it: Explain before implementing

What would be lost by replacing the command-side aggregate with a writable DTO?

STEP 21

A User Wants to Schedule a Term: What Actually Happens?

The SchoolGrid situation

The administrator requests Term 2 for 1 May through 28 July 2026. A domain operation is necessary, but it is not the entire user interaction.

Ideas to develop

Sketch request handling: interpret input, locate the academic year, ask it to schedule the term, persist an accepted change, and report the outcome. Identify missing-year, rejected-rule, and save-failure paths. Avoid teaching an HTTP endpoint as though it were the business operation itself.

Example: The example to build on

Given the 2026 academic year exists with a compatible Term 1.

When the user requests the proposed Term 2 period.

Then the use case reports success only after the accepted schedule has been saved.

Testing checkpoint

Write an application-level acceptance example before introducing its service implementation.

Try it: Explain before implementing

Which steps decide business acceptability, and which merely arrange for that decision to occur?

READING CUE

What you will discover

Trace one use case and distinguish user intention, coordination, domain decision, and persistence.

STEP 22

Application Services: Coordinating a Use Case

READING CUE

What you will discover

Separate orchestration from domain policy and from infrastructure implementation.

The SchoolGrid situation

The same scheduling workflow should work whether the user starts it from a form or a command-line interface.

Ideas to develop

Give an application service responsibility for coordinating loading, domain operations, persistence, and result production. Keep the non-overlap decision in the domain. Discuss transaction coordination and access checks at the appropriate use-case boundary without claiming every form of authorisation is non-domain logic.

Example: The example to build on

Given the requested academic year cannot be found.

When the scheduling use case runs.

Then it returns an appropriate missing-year outcome without attempting to schedule or save a term.

Testing checkpoint

Use a repository fake to test success, missing state, domain rejection, and persistence failure separately.

Try it: Explain before implementing

How would duplicated overlap checks in multiple application services weaken the design?

STEP 23

Commands and Use-Case Input Models

The SchoolGrid situation

Several interfaces must convey the same intention: schedule Term 2 in the 2026 academic year for a proposed period.

Ideas to develop

Introduce `ScheduleTermCommand` as a use-case input contract. Discuss required fields, parsed dates, and permitted values at their respective boundaries. A command asks for a change; its existence does not prove that the change is allowed. Do not require a command bus, mediator, or event-sourced system.

Example: The example to build on

Given a well-formed command contains dates that overlap Term 1.

When the use case processes it.

Then the domain still rejects the requested change despite the command's valid shape.

Testing checkpoint

Test malformed transport input separately from a well-formed but unacceptable scheduling request.

Try it: Explain before implementing

Why is `TermRescheduled` a different kind of message from `RescheduleTerm`?

READING CUE

What you will discover

Explain command intent and distinguish request shape from domain validity.

STEP 24

DTOs: Moving Data Across Boundaries

READING CUE

What you will discover

Recognise DTOs and projections and explain why transport data should not be mistaken for a domain model.

The SchoolGrid situation

The interface needs a predictable response containing the year, term number, and displayed dates, not access to a mutable domain object.

Ideas to develop

Explain a DTO's data-transfer purpose, mapping, and serialisation. A projection into an agreed data structure can be a DTO; selecting attributes alone does not make it an entity. Contrast boundary data with domain behaviour and consider how clients evolve without exposing internal object structure.

Example: The example to build on

Given a calendar summary has been produced.

When a client changes its local copy of the response.

Then the authoritative calendar does not change merely because response data was edited.

Testing checkpoint

Check response fields and mapping independently from domain scheduling tests.

Try it: Explain before implementing

Why should a client not be able to bypass business rules by sending an edited response object back?

STEP 25

Queries and Read Models

The SchoolGrid situation

The administrator opens a dashboard listing the dates of every scheduled term. No scheduling decision is being requested.

Ideas to develop

Develop a calendar-summary read model and projection. Separate reads from changes conceptually without demanding separate databases or full CQRS infrastructure. Discuss consistency expectations, access filtering, and pagination only where the actual screen needs them. A read projection cannot authorise a later change by itself.

Example: The example to build on

Given a year contains two scheduled terms.

When the summary is requested.

Then the read result shows those two terms accurately without inventing a missing third term.

Testing checkpoint

Test ordering, empty years, partial schedules, and the declared consistency expectation of the chosen read path.

Try it: Explain before implementing

When is loading an aggregate the simplest read implementation, and when is a projection clearer?

READING CUE

What you will discover

Choose an appropriate read path and explain when aggregate reconstruction is unnecessary.

STEP 26

The Architecture Problem: Our Domain Now Needs a Database and UI

READING CUE

What you will discover

Identify coupling pressures before introducing architecture layers.

The SchoolGrid situation

The calendar rules now need to be reached through a user interface and stored between sessions. Technical choices start pulling on the model.

Ideas to develop

Compare putting SQL, HTTP parsing, and calendar decisions into one operation with separating their responsibilities. Ask what should remain stable when an interface or persistence mechanism changes. Keep architecture proportional to the project: the problem is unwanted dependency, not an insufficient number of folders.

Example: The example to build on

Given the school replaces its command-line interface with a web form.

When the same scheduling request is submitted.

Then its business acceptability should remain the same.

Testing checkpoint

Retain the existing domain tests while experimenting with different adapters.

Try it: Explain before implementing

Which changes should require revisiting the school's rules, and which should not?

STEP 27

Clean Architecture: Protect the Business Rules

The SchoolGrid situation

A web-framework upgrade should not change whether two school terms may share a day.

Ideas to develop

Introduce the dependency rule and a practical domain, application, infrastructure, and presentation arrangement. Distinguish source-code dependency from runtime collaboration. Present the four layers as this book's mapping, not the only valid number or naming of Clean Architecture boundaries.

Example: The example to build on

Given the database implementation changes.

When the same calendar examples are evaluated.

Then the meaning of valid and invalid schedules remains independent of that technology change.

Testing checkpoint

Use unchanged domain tests as evidence of behavioural stability; separately inspect dependencies.

Try it: Explain before implementing

Can a project have four correctly named folders while still violating the intended dependency direction?

READING CUE

What you will discover

Explain business-policy independence and distinguish architectural intent from a fixed folder recipe.

STEP 28

The Domain Layer

READING CUE

What you will discover

Locate domain responsibilities and justify the placement of inward-facing contracts.

The SchoolGrid situation

The team sorts its calendar concepts by what they mean rather than by which screen uses them.

Ideas to develop

Place value objects, entities, aggregate behaviour, and genuinely needed domain services, events, and factories with domain policy. For this teaching design, locate the aggregate repository contract inward with the domain; recognise that other architectures put such a port in the application layer. Keep database implementations outside.

Example: The example to build on

Given a new UI framework is introduced.

When the domain package is imported and its examples are exercised.

Then the domain does not require that framework to understand a calendar.

Testing checkpoint

Check both business behaviour and the absence of outward framework imports.

Try it: Explain before implementing

Does every domain need all of the tactical patterns listed in a textbook?

STEP 29

The Application Layer

The SchoolGrid situation

Scheduling a term requires a sequence of work, but the calendar should not know which controller requested it.

Ideas to develop

Collect application services, commands, use-case DTOs, and inward ports around user goals. Discuss not-found outcomes, transaction coordination, and authorisation entry points while keeping domain rules in the model. Contrast application-specific workflow with transport-specific concerns such as HTTP status codes.

Example: The example to build on

Given the calendar rejects overlapping dates.

When the application handles that outcome.

Then it does not save a successful change or decide its own alternative overlap policy.

Testing checkpoint

Test orchestration using real domain objects and substituted external ports.

Try it: Explain before implementing

Should the application use case return an HTTP response object to a command-line caller?

READING CUE

What you will discover

Locate use cases, input/output contracts, coordination, and application-specific outcomes.

STEP 30

The Infrastructure Layer

READING CUE

What you will discover

Identify technical adapters and distinguish their behaviour from business-policy decisions.

The SchoolGrid situation

The application needs the year retrieved and saved. Somebody must translate that request into database operations.

Ideas to develop

Place repository implementations, database mapping, transaction mechanisms, and external-service adapters here. Discuss event delivery only when the optional event requirement is enabled. Explain how storage constraints and version checks support the domain rather than replace its expressive rules.

Example: The example to build on

Given a valid academic year is saved and later loaded.

When the infrastructure reconstructs the aggregate.

Then its terms and date values retain the intended meaning without treating loading as a new scheduling event.

Testing checkpoint

Plan integration tests for round trips, constraints, concurrency conflicts, and rollback behaviour.

Try it: Explain before implementing

Why can a perfect in-memory fake still fail to reveal an incorrect database mapping?

STEP 31

The Presentation Layer

The SchoolGrid situation

A web request contains text dates, while a command-line command contains arguments. Both express the same calendar intention.

Ideas to develop

Translate transport input into application input and translate application results back to the caller. Address date parsing, request shape, and useful error messages. Keep overlap logic out of controllers. Explain that a frontend may offer helpful early validation without becoming the authoritative domain boundary.

Example: The example to build on

Given an incoming date cannot be parsed.

When the presentation adapter reads the request.

Then it reports malformed input rather than constructing a misleading scheduling command.

Testing checkpoint

Test transport parsing and outcome translation separately from domain rule enforcement.

Try it: Explain before implementing

Why must the system still enforce rules when a user bypasses the web form?

READING CUE

What you will discover

Explain inbound adapters and distinguish transport validation from scheduling decisions.

STEP 32

The Dependency Rule

READING CUE

What you will discover

Read dependency arrows correctly and separate imports from execution order.

The SchoolGrid situation

The application calls a repository during execution. Does that mean its source code must import a database repository class?

Ideas to develop

Draw source dependencies toward inner policies and distinguish them from a runtime sequence diagram. Locate concrete object assembly at the outer composition boundary. Show how interfaces allow an inner use case to call outward without knowing outward implementation details.

Example: The example to build on

Given the application needs an AcademicYear repository.

When its source dependencies are inspected.

Then it refers to an inward contract rather than importing a concrete SQL adapter.

Testing checkpoint

Add a dependency check or focused review that detects an inward module importing infrastructure.

Try it: Explain before implementing

Can a runtime call point outward while a source-code dependency still points inward?

STEP 33

Dependency Inversion Through the Repository

The SchoolGrid situation

The scheduling use case should collaborate with a repository without constructing a particular database implementation.

Ideas to develop

Develop the relation Application to abstraction, with Infrastructure implementing that abstraction. Explain injecting a concrete collaborator at assembly time. Distinguish the architectural inversion from the technique used to provide an object; injecting a concrete database dependency alone does not ensure inversion.

Example: The example to build on

Given a use case accepts a repository contract.

When the composition root supplies either an in-memory or a database implementation.

Then the use-case logic does not need to change its scheduling policy.

Testing checkpoint

Run the same application scenarios with a fake, then check the real adapter against the required contract.

Try it: Explain before implementing

Why is constructor injection not the same thing as an entire dependency-inversion design?

READING CUE

What you will discover

Explain dependency inversion, dependency injection, and the composition root using one repository.

STEP 34

Why the Domain Does Not Know About the Database

READING CUE

What you will discover

Separate domain design from persistence mapping and evaluate tradeoffs without absolutism.

The SchoolGrid situation

A term's identity and an academic year's rules must make sense before the team chooses tables or an ORM.

Ideas to develop

Compare persistence-independent domain objects with an Active Record-style design. Discuss simple CRUD situations fairly, then explain why this teaching model isolates policy from storage. Address identity mapping, reconstitution, and the difference between a persisted object and a DDD entity.

Example: The example to build on

Given the repository reloads an existing academic year.

When the application asks it a scheduling question.

Then the answer follows domain rules rather than being defined by a database row's method set.

Testing checkpoint

Keep domain tests free of database setup; test mapping and persistence semantics in integration.

Try it: Explain before implementing

Would removing ORM annotations automatically produce a well-modelled domain?

STEP 35

Walking Through the Entire Request

The SchoolGrid situation

The administrator reschedules Term 2 to start on 15 April. We now have names for every responsibility involved.

Ideas to develop

Trace presentation to application, repository loading, aggregate decision, persistence commit, result mapping, and presentation response. Contrast the runtime path with the inward dependency diagram. Include missing-year, invalid-period, overlap, concurrency conflict, and commit-failure outcomes without reporting success prematurely.

Example: The example to build on

Given Term 2's proposed dates pass all domain rules.

When the persistence commit fails.

Then the caller is not told that the new calendar was successfully saved.

Testing checkpoint

Create a sequence-based test plan and distinguish unit-level failure simulations from real transaction tests.

Try it: Explain before implementing

At what exact point is the system entitled to report that the reschedule succeeded?

READING CUE

What you will discover

Follow request, success, rejection, and commit failure across the architecture.

STEP 36

Testing Value Objects

READING CUE

What you will discover

Develop focused value-object tests and recognise boundaries and useful properties.

The SchoolGrid situation

A small error in inclusive dates can affect every term, so we revisit the smallest model with systematic examples.

Ideas to develop

Extend the earlier TDD cycles with equality, immutability, inclusion, duration, overlap, and invalid replacements. Introduce property-based thinking without requiring a new library. Examples include overlap symmetry and a same-day range covering one day. Test behaviour, not the decorator or internal representation used to implement it.

Example: The example to build on

Given two ranges are compared for overlap.

When their argument order is reversed.

Then the overlap result remains the same.

Testing checkpoint

Select one missing example, see the relevant failure, implement it, and refactor; do not retrospectively relabel all earlier tests as TDD.

Try it: Explain before implementing

Which boundaries would you test before trusting an inclusive duration calculation?

STEP 37

Testing Entities and Aggregates

The SchoolGrid situation

The administrator makes several calendar changes. Every accepted state must remain coherent, including after an unsuccessful request.

Ideas to develop

Grow tests for scheduling, duplicate rejection, rescheduling, cross-year dates, term order, and failed changes. Use business language and observable state. Distinguish a complete calendar fixture from legal partial states, and test optional events only when that extension is enabled.

Example: The example to build on

Given Term 2 has an accepted period.

When a replacement starts on Term 1's last day.

Then the operation rejects it and retains the original term identity and period.

Testing checkpoint

Use arrange–act–assert to organise each example; explain that this structure is different from the red–green–refactor development cycle.

Try it: Explain before implementing

Why should a rejection test verify the remaining state instead of only checking that an error occurred?

READING CUE

What you will discover

Verify identity, shared invariants, and unchanged state after rejected operations.

STEP 38

Testing Application Use Cases

READING CUE

What you will discover

Use appropriate test doubles while retaining real domain behaviour in use-case tests.

The SchoolGrid situation

We want to test the scheduling workflow without starting a database for every business example.

Ideas to develop

Introduce a repository fake, distinguish it from a stub or mock, and avoid replacing the aggregate with a mock that cannot enforce its rules. Test success, not found, domain rejection, and failed persistence. Explain the need for contract tests so a convenient fake does not promise behaviour the real repository lacks.

Example: The example to build on

Given the fake repository contains the 2026 academic year.

When the application receives overlapping Term 2 dates.

Then it reports rejection without recording a successful new schedule.

Testing checkpoint

Prefer observable use-case outcomes to brittle assertions about every internal method call.

Try it: Explain before implementing

What bug could be hidden by mocking the aggregate's scheduling decision to always succeed?

STEP 39

Testing Infrastructure Separately

The SchoolGrid situation

Two administrators load the same calendar version and both attempt to save. An in-memory business-rule test cannot establish the database outcome.

Ideas to develop

Plan isolated integration tests against the actual persistence technology: round trips, unique identities, version conflicts, rollback, and transaction boundaries. Where events are enabled, test outbox persistence and retry safety. Keep setup deterministic and distinguish these tests from a fake-only application test.

Example: The example to build on

Given two sessions load the same academic year version.

When one commits and the other submits a stale save.

Then the chosen concurrency strategy prevents silently overwriting the committed change.

Testing checkpoint

Reproduce the competing saves with independent sessions and verify the final stored state, not only the exception.

Try it: Explain before implementing

Which infrastructure guarantees are impossible to prove with only a repository fake?

READING CUE

What you will discover

Explain why mapping, transactions, concurrency, and delivery require real integration evidence.

STEP 40

The Different Tests Answer Different Questions

READING CUE

What you will discover

Distinguish domain, application, integration, and end-to-end tests and use each deliberately.

The SchoolGrid situation

A correct date calculation, a correct workflow, and a working deployed request are related but different claims.

Ideas to develop

Map risks to test boundaries: domain rules, use-case coordination, adapter behaviour, and an assembled user journey. Explain why higher-level tests do not replace focused examples and why terminology varies between teams. Revisit TDD as a way of developing behaviour, not a synonym for a particular test layer.

Example: The example to build on

Given the domain correctly rejects an overlap.

When the web adapter mistakenly reports success.

Then domain tests can pass while a presentation or end-to-end check reveals the integration defect.

Testing checkpoint

Assign each major failure mode to the narrowest test that can expose it, with a small number of complete journey tests.

Try it: Explain before implementing

Which test would first reveal a broken JSON field name, and which would reveal a shared-day overlap bug?

STEP 41

One Business Rule Through Every Layer

The SchoolGrid situation

Return to the original conflict: Term 1 ends on 28 March and the requested Term 2 start is also 28 March.

Ideas to develop

Follow the rule through a stakeholder statement, boundary example, failing test, aggregate decision, application outcome, and useful user message. Include the role of storage consistency without duplicating domain policy everywhere. Distinguish where a rule is authoritative from the many places its outcome is represented.

Example: The example to build on

Given Term 1 includes 28 March 2026.

When the user requests that same date for Term 2's start.

Then the change is rejected, nothing invalid is persisted, and the caller receives a meaningful explanation.

Testing checkpoint

Build a traceability chain linking the rule to domain, application, and selected end-to-end evidence.

Try it: Explain before implementing

Does tracing one rule through several layers mean each layer should independently implement it?

READING CUE

What you will discover

Trace one policy from conversation to model, execution, persistence, and feedback.

STEP 42

DDD vs Clean Architecture vs TDD

READING CUE

What you will discover

Explain the distinct contributions of modelling, dependency design, and test-driven development.

The SchoolGrid situation

The team has used three approaches in one project. Students need to explain their relationship without treating them as interchangeable labels.

Ideas to develop

Compare DDD's model and language, Clean Architecture's dependency boundaries, and TDD's incremental feedback cycle. Use the same rescheduling example for all three. Discuss how each can be practised without the others and why their combination still requires judgement and conversations about the domain.

Example: The example to build on

Given an overlap example reveals a missing rule.

When the team develops and places the resulting behaviour.

Then DDD informs meaning, TDD guides the small development steps, and architecture protects the policy's dependencies.

Testing checkpoint

Ask students to identify which concern each design decision addresses and what evidence would support it.

Try it: Explain before implementing

Could a thoroughly tested four-layer application still misunderstand the school's calendar?

STEP 43

Common Misunderstandings

The SchoolGrid situation

A student proposes a repository for every class, a service for every operation, and microservices because the project uses DDD.

Ideas to develop

Revisit class versus entity, entity versus aggregate root, value object versus identifier, DTO versus domain object, and domain service versus application service. Address repository and projection boundaries, legitimate application logic, optional patterns, and the fact that folders or microservices do not establish DDD or Clean Architecture.

Example: The example to build on

Given `DateRange` is a useful domain class.

When someone concludes it needs an identity and repository.

Then the team asks what business requirement would justify those additions rather than accepting the checklist.

Testing checkpoint

Use counterexamples from earlier steps and ask which existing tests would expose a harmful redesign.

Try it: Explain before implementing

Which abstraction would you deliberately omit from the smallest useful version of SchoolGrid, and why?

READING CUE

What you will discover

Challenge pattern checklists and justify decisions using the SchoolGrid rules.

STEP 44

The Complete SchoolGrid Mental Model

READING CUE

What you will discover

Explain the entire design as a connected response to the school's needs and evaluate a new rule.

The SchoolGrid situation

The administrator asks one final question: how does the system let us change dates while keeping the calendar trustworthy?

Ideas to develop

Connect values, identity, aggregate boundaries, use cases, repositories, DTOs, adapters, dependency direction, and the test strategy in one account. Mark services, dedicated factories, and external events as choices justified by extensions, not mandatory decorations. End with a fresh business change rather than a catalogue of definitions.

Example: The example to build on

Given the school proposes allowing a term to cross into the next calendar year.

When students analyse the change before modifying code.

Then they identify the affected policy, examples, and model assumptions without automatically redesigning every layer.

Testing checkpoint

Write the next failing example, identify the responsible model boundary, and explain which regression and integration tests still matter.

Try it: Explain before implementing

Can you explain every major abstraction by naming the problem it solves and one simpler alternative?

Author's checklist

- Does the section begin with a school problem rather than a pattern definition?
- Is each policy agreed for this teaching school or explicitly hypothetical?
- Do examples cover acceptance, rejection, and significant boundaries?
- Is each responsibility justified before its technical name is introduced?
- Are test-first development and later regression checking distinguished?
- Are optional abstractions and credible simpler alternatives visible?
- Does the next section follow naturally from an unresolved question?

Authoring with the teaching toolkit

All ten additions are reusable. The standalone [student example booklet](#) and [instructor example booklet](#) demonstrate them using a reservation story. This section shows the authoring syntax.

— Connect evidence to rules

```
1 \begin{conversation}{When does a hold expire?}
2 \utterance{msg:time}{Business expert}{After 48 hours.}
3 \end{conversation}
4 \begin{businessrule}{br:hold}{Limit the hold}
5 A reservation lasts 48 hours.
6 \evidence{\msgref{msg:time}.}
7 \end{businessrule}
```

Use unique label keys. Message and rule numbers follow the step automatically. Use `\msgref` and `\ruleref` for clickable references.

— Make a model revision visible

```
1 \begin{modelevolution}{Clarify identity}
2 \BeforeModel{One flag per title.}
3 \NewEvidence{Copies have separate barcodes.}
4 \AfterModel{Use \modelchange{Copy identity}.}
5 \end{modelevolution}
6 \begin{misconception}{Storage implies identity}
7 \Mistaken{Every stored row is a domain entity.}
8 \Reconsider{Ask what identity means to the business.}
9 \end{misconception}
```

For SchoolGrid, the same pattern can show the shift from editing dates independently to validating the calendar as a whole. Label hypothetical changes explicitly.

Accept, test, and link the implementation

```

1 \begin{acceptancetest}{at:boundary}{At expiry}{\ruleref{br:hold}}
2 \Given{an existing reservation ending at 09:00.}
3 \When{a member requests the copy at exactly 09:00.}
4 \Then{the request succeeds.}
5 \automatedby{CopyExamples.acceptsAtExactExpiry()}
6 \end{acceptancetest}
7 \testref{at:boundary}
8 \begin{companioncode}{Example}{snapshot-v1}
9 \companionlink{code/Copy.java}{Open domain source}
10 \end{companioncode}

```

External code listings print the source that actually runs. A checksum manifest identifies the distributed snapshot. Local file links need the code folder beside the PDF; replace their target with a real hosted URL when you publish one.

Add behaviour diagrams

```

1 \begin{figure}[H]
2 \centering
3 \input{diagrams/reservation-sequence}
4 \caption{A request passing through the model.}
5 \end{figure}
6 % Also available: diagrams/reservation-state.tex

```

Use the supplied TikZ styles for participants, lifelines, messages, returns, states, and transitions. Keep class diagrams for structure, sequence diagrams for collaborations, and state diagrams for lifecycle behavior.

Close the learning loop

```

1 \begin{checkpoint}
2 \checkpointitem{Rules agreed}{Name the rules.}
3 \checkpointitem{Explain}{Justify the model boundary.}
4 \checkpointitem{Unresolved}{Record the next question.}
5 \end{checkpoint}

```

— One source, two editions

```

1 \begin{solution}{Exercise title}
2 Explain the reasoning behind a possible solution.
3 \end{solution}
4 \begin{instructornote}{Teaching prompt}
5 Add a discussion question or assessment rubric.
6 \end{instructornote}

```

Both environments are omitted entirely in a student build. The wrapper selects the instructor edition before loading the manuscript. Use an external `\lstinputlisting` for code inside hidden content; inline verbatim listings are not supported in these captured environments.

— Define once, find everywhere

```

1 % In glossary.tex:
2 \DeclareDDDConcept{entity}{Entity}{An object with identity.}
3 % In ordinary prose:
4 \dddtterm{entity} % glossary link and index entry
5 \dddnote{entity} % margin definition and index entry
6 % In the back-matter glossary:
7 \glossaryentry{entity}
8 % For any additional index topic:
9 \index{UML!sequence diagram}

```

Definitions in `glossary.tex` are shared by the margin and glossary. Add new entries to the printed glossary in alphabetical order. Each use of `\dddtterm` or `\dddnote` records its page in the index. Plain, unmarked occurrences are not indexed automatically. Examples: `Entity`, `Aggregate`, `Value object`.

— Cite and build

Use `\cite{evans2015}` to produce a reference such as Evans [2]. Add bibliography entries to `references.bib`. The Gherkin example follows Cucumber's reference [1]; Fowler gives additional context for bounded contexts [3].

Run `.\build.ps1` to compile the example editions, bibliography, index, and runnable code. Add `-Target Book` to build SchoolGrid, or `-Target All` for both. The script uses LuaLaTeX, BibTeX, and MakeIndex.

DDD glossary

These short teaching definitions are used consistently in margin notes and prose. For the broader pattern vocabulary, consult Evans [2].

Aggregate

A consistency boundary around domain objects. Its root controls changes and protects invariants.

Aggregate root

The entity through which external callers access and change an aggregate.

Bounded context

A boundary within which one domain model and its language have a consistent meaning.

Domain event

An immutable record of a domain-significant fact that has already happened.

Entity

An object with a continuing identity, even when its attributes change.

Factory

An operation or object that creates valid domain objects while hiding construction details.

Invariant

A condition the model must preserve at its consistency boundary.

Repository

A collection-like interface for retrieving and persisting aggregate roots while hiding storage details.

Ubiquitous language

Shared domain language used by experts and developers within a bounded context.

Value object

An immutable description defined by its values, with no independent identity.

References and further reading

- [1] Cucumber. Gherkin reference. <https://cucumber.io/docs/gherkin/reference/>. Accessed 7 September 2026.
- [2] Eric Evans. Domain-driven design reference: Definitions and pattern summaries. Domain Language. <https://www.domainlanguage.com/ddd/reference/>, 2015. Accessed 7 September 2026.
- [3] Martin Fowler. Bounded context. <https://martinfowler.com/bliki/BoundedContext.html>, 2014. 15 January 2014. Accessed 7 September 2026.

Concept index

Aggregate, 83, 126

Aggregate root, 91

Bounded context, 17

Domain event, 96

Entity, 126

Factory, 95

Invariant, 92

Repository, 97

Ubiquitous language, 17

Value object, 33, 126